RTL Design Sherpa

CELL Micro-Architecture Specification 0.12

October 19, 2026

Table of Contents

1 Cell Mas Index....12

2 Cell — Compute Engine IP Product Requirements Document....13

2.1 1. Identity and scope....13

2.2 2. Architecture....15

2.2.1 Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right)....15

2.3 3. Dataflow....17

2.4 4. ALU and execution FSM....18

2.5 5. Packet interface....19

2.6 6. Credit behavior....19

2.6.1 6.1 Boot advertisement....19

2.6.2 6.2 Consumed-line credit returns....20

2.6.3 6.3 Straps on the result path....20

2.7 7. Config registers....21

2.8 8. Status and observability....23

2.9 9. Verification hooks....23

2.10 10. Dependencies....24

2.11 11. Deferred....24

2.12 12. Open questions....25

2.13 References....25

3 Tile — Compute Engine Tile Subsystem Specification....27

3.1 1. Scope....27

3.2 2. Hierarchy..... 28
3.3 3. Connectivity.....29
3.4 4. Modules.....29
3.5 5. Observation (MonBus aggregation).....30
3.6 6. Interfaces.....31
3.7 7. Modes.....31
3.8 8. Rapids attachment.....32
3.9 9. Clocking and reset.....32
3.10 10. Open items.....32
3.11 11. Hierarchical tile, root nodes, addressing (ADOPTED 2026-10-16).....33
3.11.1 11.1 Hierarchy.....34
3.11.2 11.2 Payload types.....34
3.11.3 11.3 Destination coordinate.....35
3.11.4 11.4 Root failover.....36
3.11.5 11.5 Packet header.....36
3.11.6 11.6 Transport.....38
3.11.7 11.7 Where the fields come from.....39
3.11.8 11.8 Operations.....39
3.11.9 11.9 End-to-end credits.....40
3.11.10 11.10 Credit advertisement.....43
3.11.11 11.11 Cell-issued descriptors.....44
3.11.12 11.12 RAPIDS implementation status (2026-10-17).....45
3.11.13 11.13 Tetrad control core (owner direction 2026-10-17; deferred 2026-10-18; ruling 2026-10-19).....47

3.11.14 11.14 Result flush to the tile I/O (owner direction 2026-10-18; defined 2026-10-19)....48
3.11.15 11.15 Multi-path routing and failover: open hazards (2026-10-18)....49
3.12 12. Milestones....50
3.13 References....50
4 ALU-INFER — Cell Inference ALU Product Requirements Document....52
4.1 1. Identity and scope....52
4.2 2. Element model....53
4.3 3. Function set (v1)....54
4.3.1 Function-set roadmap (owner ruling 2026-10-19)....56
4.4 4. Interface contract....57
4.5 5. Control interface — one RV32I core per tetrad, in-band everything....60
4.6 6. Verification hooks....61
4.7 7. Deferred and growth....62
4.8 8. Open questions....63
4.9 9. Dependencies....63
4.10 10. References....64
5 ALU-INFER Micro-Architecture Specification Index....66
5.1 Overview....66
5.1.1 Key Features....66
5.1.2 Design Philosophy....67
5.2 Design Notes....67
5.2.1 Version History....67
5.3 References....68

5.3.1 Visual Assets....68
5.3.2 Companion Specifications....69
5.3.3 Project-Level....69
5.4 Navigation....69
5.4.1 Document Organization....69
5.4.2 Quick Navigation....70
6 Document Information....71
6.1 Overview....71
6.1.1 CELL Micro-Architecture Specification....71
6.1.2 Document Purpose....71
6.1.3 Intended Audience....71
6.2 Design Notes....72
6.2.1 Revision History....72
6.3 References....74
6.3.1 Related Documents....74
7 Design Philosophy and Scope....75
7.1 What This Block Is....75
7.2 Scope and Deferrals....75
7.3 MAS Decisions at a Glance....76
7.4 Relationship to the ALU....77
8 Module Tree....79
8.1 Purpose....79
8.2 Module Tree....79
8.3 Block Intent (one paragraph each)....79

8.3.1 delta_mesh_cell....79
8.3.2 cell_rx....79
8.3.3 cell_cfg_regs....80
8.3.4 cell_mem_in....80
8.3.5 cell_mem_out....80
8.3.6 cell_credit....80
8.3.7 cell_packetizer....80
8.3.8 cell_exec_fsm....80
8.4 Parameters....81
8.5 Clock and Reset....81
9 Datapath Figures....83
9.1 Whole-Cell Datapath....83
9.1.1 Figure 2.1: Whole-cell datapath....83
9.2 Credit Loop....85
9.2.1 Figure 2.2: Credit loop....85
9.3 ALU Datapath Focus....86
9.3.1 Figure 2.3: ALU datapath focus....86
10 External and Internal Interfaces....88
10.1 Cell Boundary....88
10.1.1 Mesh-Facing Ports....88
10.1.2 Local Port Contract....89
10.1.3 AXI4-Lite Register Interface....89
10.1.4 Observability....89
10.2 Internal Interfaces....90

10.2.1 cell_rx → cell_mem_in....90
10.2.2 cell_rx → cell_cfg_regs....90
10.2.3 cell_cfg_regs → datapath....91
10.2.4 cell_mem_in → cell_exec_fsm....92
10.2.5 cell_exec_fsm → cell_mem_out....92
10.2.6 cell_exec_fsm → cell_credit....92
10.2.7 cell_credit → node (credit VC)....93
10.2.8 cell_mem_out → cell_packetizer....93
10.2.9 cell_packetizer → node (data network)....93
10.2.10 cell_exec_fsm ↔️ ALU....93
11 Cycle Behavior: Fill Path as Built....94
11.1 Slice-1 Fill Path....94
11.1.1 Head-Beat Reception....94
11.1.2 Coordinate Match....94
11.1.3 Demux Decision....94
11.1.4 Payload Write Timing....95
11.1.5 Occupancy Accounting....95
11.2 Expected Slice-2/3 Cycle Behavior....95
12 Credit Logic....97
12.1 MAS Decision: Memory Sizing and Advertisement....97
12.2 MAS Decision: Credit Batch Size and Timeout....97
12.3 Per-Source Owed Counters....97
12.4 Credit Packet Format....98
12.5 Boot Advertisement Packet....98

12.6 Sender-Generic Returns....99
12.7 Implementation Note: Counter Bank Sizing....99
13 Packetizer and Execution FSM....100
13.1 Packetizer....100
13.1.1 MAS Decision: Batching and Flush Conditions....100
13.1.2 MAS Decision: Strap Counter Initialization....100
13.1.3 Header Build....100
13.1.4 Destination and Tag Programming....101
13.1.5 Strap Accounting....101
13.2 Execution FSM....102
13.2.1 Pop → Issue → Push Loop....102
13.2.2 MAS Decision: Operand-Source Mux....102
13.2.3 IP/OP Pointer Behavior....102
13.2.4 Gatling as the Cell ALU Sequencer (owner ruling 2026-10-19)....103
13.2.5 ALU Hookup....103
13.2.6 Issue Tag and Same-Tag Alignment (ratified spec — RTL pending)…104
13.2.7 MAC Accumulation....104
13.2.8 Soft Reset....104
14 Verification Plan....105
14.1 House Test Grid....105
14.2 Existing Tests....105
14.3 Planned Slice-2 Credit-Loop Test....105
14.4 Planned Slice-3 ALU-Run Test....106
14.5 Verification Hooks....106

14.6 Assertions and Checks....107
15 Register Map and MAS Decisions....108
15.1 Register Map....108
15.2 Bit Fields....109
15.2.1 CTRL (0x00)....109
15.2.2 STATUS (0x04) — read-only....110
15.2.3 FUNC_SEL (0x08)....110
15.2.4 OP_SRC (0x34)....110
15.2.5 IRQ_STATUS / IRQ_EN....111
15.3 MAS Decisions Revisited....111
15.4 RTL Naming Divergence (slice 1)....112
16 References....113
16.1 Normative Specifications....113
16.2 RTL Sources....113
16.3 Verification....114
16.4 House Style and Build....114

List of Figures

Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right).....15
Figure 2.1: Whole-cell datapath.....83
Figure 2.2: Credit loop.....85
Figure 2.3: ALU datapath focus.....86

List of Tables

Table 1: MonBus sources per level....30
Table 2: Proposed destination coordinate, {ext, level, X, Y, Z}, 15 bits at COORD_W = 4....35
Table 3: Proposed tile packet header fields (beyond delta's existing ones)....36
Table 4: Proposed header packing....37
Table 5: Proposed operations between RAPIDS and the tile....39
Table 6: Credits toward RAPIDS....42
Table 7: Credit packet operations....43
Table 8: Document information....71
Table 9: Revision history....72
Table 10: Table 1.1: Summary of MAS decisions....76
Table 11: Table 2.1: cell_top parameters....81
Table 12: Table 3.1: Mesh-facing AXI4-Stream ports....88
Table 13: Table 3.2: Observability outputs....89
Table 14: Table 3.3: Input-memory write port....90
Table 15: Table 3.4: Config-stream register interface....90
Table 16: Table 3.5: Register outputs to datapath....91
Table 17: Table 3.6: Input-memory read port....92
Table 18: Table 3.7: Result-memory write port....92
Table 19: Table 3.8: Consumed-line credit strobe....92
Table 20: Table 3.9: Result-memory read port....93
Table 21: Table 4.1: Slice-1 head-beat dispatch....94
Table 22: Table 4.2: Result-packet header fields....100
Table 23: Table 4.3: OP_SRC encoding....102
Table 24: Table 4.4: ALU↔FSM seam signals....103
Table 25: Table 5.1: House test grid....105
Table 26: Table 5.2: Verification hooks....106
Table 27: Table 6.1: Cell register map....108
Table 28: Table 6.2: CTRL bit fields....109
Table 29: Table 6.3: STATUS bit fields....110
Table 30: Table 6.4: FUNC_SEL fields....110
Table 31: Table 6.5: OP_SRC fields....110
Table 32: Table 6.6: IRQ fields....111
Table 33: Table 6.7: MAS decisions and their hardware homes....111

1 Cell Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

← Back to cell

2 Cell — Compute Engine IP Product Requirements Document

Version: 0.5 (2026-10-19): records the 2026-10-19 owner ruling that **gatling sequences the cell ALU** (`POP`/`PUSH`/`FLUSH` Xcell encodings, gatling PRD v0.2) while the slice-3 `cell_exec_fsm` remains the standalone bring-up driver; `DEST` is explicitly reserved for the deferred cell→cell path and `FLUSH_DEST`/`FLUSH_TAG` are the operational drain destination; strap terminology replaces remaining grant terms in §§6.3/7/10 (slice-1 RTL still uses the old grant identifiers, noted in the MAS). Carries v0.4 (2026-10-18): §6.3 rewritten to the strap model (`TILE_LAND_FLITS`, RAPIDS returns on drain; owner direction 2026-10-18) and the result destination becomes gatling-programmed (TILE_SPEC §11.14) — `DEST` chain-back superseded, register retained for the deferred cell-destined path. Carries v0.3 (2026-10-18): **cell register file** + **sender-generic credits** — owner direction 2026-10-18 defines the cell as the lowest module: the alu-infer ALU, one delta node, the config registers, the ALU registers (`A`/`B`/`C`/`D` + `T`) as persistent operands for repeated calculations over a run (`OP_SRC` operand-source mux, `IP`/`OP` memory pointers), and the data-in / data-out memories; the tetrad engine is deferred (the owner-authored gatling-rv32itetrad barrel-core PRD is the leading candidate; §10). Credit returns become sender-generic — per-source batching to the recorded source coordinate; the packetizer gains `DEST` (default: recorded source, chain-back). §4 reconciles to the alu-infer elastic valid/ready pipeline ("one-function-at-a-time" superseded). The house-style graphviz figure replaces the ASCII block diagram. Carries v0.2: ALU element model, function set, and control interface reconciled to the alu-infer PRD v0.3 (owner direction 2026-10-17; owner datapath decisions 2026-10-16). **Status:** Pre-HAS PRD. This document is the cell's product definition; the HAS/MAS follow it. The tile architecture it plugs into is the adopted hierarchy of `tile/docs/TILE_SPEC.md` §11 (owner decision 2026-10-16); the packet header, payload types, coordinate, and credit/grant protocols referenced below are defined there and are not restated except where the cell consumes them.

2.1 1. Identity and scope

The **cell** is the tile's compute unit: sixteen of them make a tile. Each cell contains exactly **one delta node** — a `delta_mesh_cell` instantiated inside the cell, its local port facing the cell's internal datapath and its seven other ports passing through to the cell boundary. That containment is the **entire** cell↔delta relationship (owner clarification 2026-10-16): the node is a black-box subcomponent governed by delta's own HAS/MAS, and everything else in the cell — the memories, the execution FSM, the ALU, the ALU registers, the packetizer, the credit logic, the registers — is the cell's own design, speaking the tile-level contracts of TILE_SPEC §11, not delta internals. The node fills the cell's input memory from delta traffic; the cell's ALU and execution FSM consume that memory and produce results; results leave as batched flits back through the node toward their destination. As each input line is consumed, a credit returns to the line's sender so it may write more.

Owner decision 2026-10-16: the cell's compute engine is the **ALU** + **execution FSM** described here. The earlier CGRA-array ambition (README, 2026-10-13) is **deferred** — §11 — and the

`axon_spec.md` sketch (SERV-based result collector) is **superseded**: the result collection, packetization, and streaming it proposed are this PRD's output path, and its interfaces section predates the adopted §11 header/credit protocols.

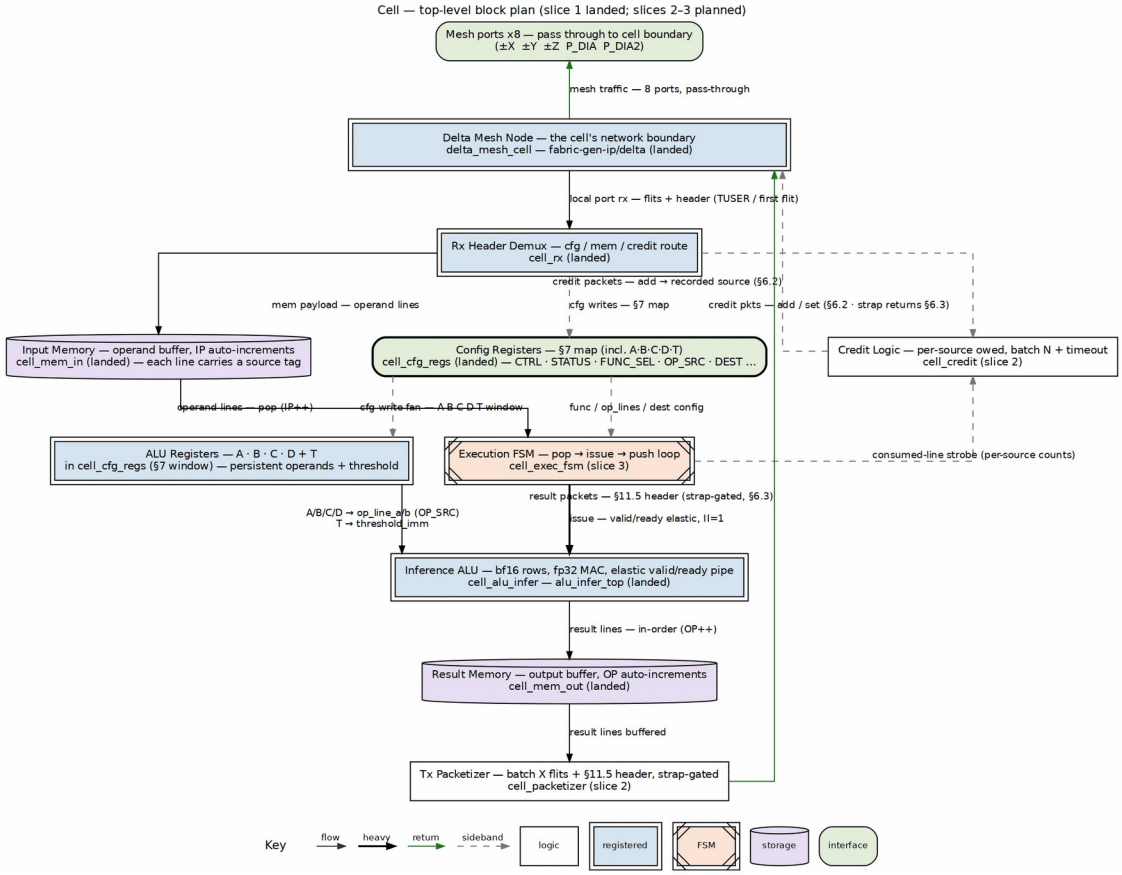
Owner direction 2026-10-17: the ALU block is governed by the `alu-infer` PRD — bf16 element model on the `rtl/math` `math_bf16_*` library, fp32 MAC accumulation, and a per-tetrad RV32I control core (not SERV; SERV is the Ocelus observability agent). Config and programming arrive in-band with the delta traffic; the tetrad core is a local reaction agent, not a config originator. See alu-infer PRD §5.

Owner direction 2026-10-18: **the cell is the lowest module and is defined first.** Its contents: the alu-infer ALU, one delta node, the config registers (§7), the ALU registers A/B/C/D (+T) — persistent operands for a sequence of the same calculations over a run — the data-in memory, and the data-out memory. The tetrad-level engine is **deferred**: gatling/tetrad is figured out once the cell is fully defined and working, with the owner-authored `gatling-rv32itetrad` barrel-core PRD (2026-10-17) the leading candidate. See §10.

Out of scope: tetrad/root node internals, rapids internals, the CXS modules themselves, and fabric-level Z stacking.

2.2 2. Architecture

2.2.1 Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right)



Cell top-level block plan

Block	Function	Notes
`delta_mesh_cell`	network attachment	instantiated black box, not owned, here — containment is the whole cell↔️delta relationship (owner clarification 2026-10-16); the cell consumes its local port and passes its mesh ports through; delta HAS/MAS

Block	Function	Notes
		govern only the node
rx header/demux	parses the §11.5 tile header (first data flit) and delta's route header (TUSER on AXIS), routes payload to input memory, handles cfg writes to registers, routes credit packets to the credit logic	per §11.6 (TILE_SPEC v1.6 clarification: tile header = first data beat on both transports; cell-bound traffic rides delta QoS ≠ 0 — QoS 0 is the node's own config plane); records each packet's tile-header source coordinate
input memory	delta-written operand/input buffer	sized in lines; its flit capacity is the boot advertisement value; each line carries a source-coordinate tag for credit returns (§6.2); read pointer IP auto-increments on pop
config registers	AXIL, registers-only per house rule	§7 — CTRL/STATUS/FUNC_SEL/OP_SRC/DEST/batching knobs/IP/OP
ALU registers	A·B·C·D persistent operands + T threshold	§7; written by cfg packets or AXIL, read into the ALU operand muxes per OP_SRC; persistent across a run (owner direction 2026-10-18)
credit logic	returns credits to the recorded sender of each consumed input line	§6.2; per-source batching, N configurable, default 1
ALU + execution FSM	pops line(s), issues them to the ALU's elastic	§4; the cell's only datapath

Block	Function	Notes
	pipeline, pushes result line(s)	
result memory	buffers result lines before packetization	sized with §12; write pointer `OP` auto-increments on push
packetizer	batches result writes into X flits, builds §11.5 header (destination per §6.3 — gatling-programmed drain), sends via node	sends only against the `TILE_LAND_FLITS` strap counter (§6.3)

2.3 3. Dataflow

A run is programmed first, then fed; everything — data, config, and programming — arrives in-band (owner ruling 2026-10-17):

1. **Program.** `cfg` packets (`ptype` = `cfg`) write the §7 registers — `FUNC_SEL`, `OP_SRC`, `DEST`, the persistent operands `A`–`D`, `T`, and the batching knobs — instead of the input memory. A `cfg` read is answered with a write-format packet echoing the tag, per §11.8.
2. **Fill.** A write packet (`ptype` = `mem`, destination coordinate `{level=0, cell X, Y, Z}`, target address inside the cell's input-memory window) arrives on the node's local port — from rapids in the v1 flow, but sender-generic by construction: any node may send it. The demux writes payload lines into the input memory and tags each line with its packet's source coordinate. The sender spends one end-to-end credit per flit sent, header included, from the per-destination counter it holds for this cell (§11.9).
3. **Consume.** The execution FSM pops the configured number of input lines for its function (`IP` auto-increments), issues the invocation to the ALU, and the elastic pipeline returns result line(s) in order; the FSM pushes them into the result memory (`OP` auto-increments).
4. **Credit return.** As each input line is consumed, the credit logic owes the line's recorded sender one credit (§6.2 — per-source batching), so the sender refills the space. Credits return on consumption, not arrival (§11.9(3)).

5. **Emit.** When the packetizer has ≥ X result flits (or a flush condition fires, §6.3), it builds the §11.5 header — destination = the gatling-programmed drain destination (§6.3; the cell does not choose it) — source = this cell's external ID, `op` = 0, length = bytes that follow, tag = the sink channel gatling programmed — and sends the packet through the node, spending its `TILE_LAND_FLITS` strap counter (§6.3). Cell-destined sending awaits the tile-spec extension (§6.3).

2.4 4. ALU and execution FSM

The FSM is a pop→issue→push loop: it issues invocations into the ALU's valid/ready **elastic pipeline** (multiple invocations in flight, results retire in order — the alu-infer MAS contract), and pushes each retired result line into the result memory. Functions are register-selected, and both the function and its operand pattern come from config, so new functions are RTL table rows, not new blocks. (The v0.2 "one-function-at-a-time" phrasing is superseded by the elastic contract; owner ruling 2026-10-17, alu-infer PRD v0.4+.)

Operand sources (owner direction 2026-10-18): the ALU's `op_line_a` / `op_line_b` are driven through an operand-source mux configured by `OP_SRC` (§7): each side selects the input-memory pop or one of the persistent ALU registers `A`–`D` (32 bits each, two bf16 lanes — matching `LINE_W`), and `T` (bf16) backs `threshold_imm`. This is the "sequence of the same calculations over a run" pattern: streamed operands arrive from delta into the input memory and pop under `IP`, while persistent scalars (weights, biases, constants) sit in `A`–`D` for the whole run. `IP`/`OP` auto-increment on pop/push and wrap at the configured memory depth; `CTRL.soft_reset` clears them.

Element model (owner direction 2026-10-17, superseding the B4/B8/B16/B32 note of 2026-10-16): a 32-bit line is read as **2×bf16** elements, and the function array is built mostly from the `math_bf16_*` family of `rtl/math` (adder, multiplier, FMA, comparator, min/max, ReLU, activations as growth). The ALU's product definition — element model, function set, interface contract, verification doctrine, and the per-tetrad control interface — is the **`alu-infer` PRD**, which governs the ALU block; it is not restated here (one source per fact). MAC accumulates in fp32 with one rounding per result line at drain. Wider lines (§12) scale the lane count, never the element width.

Gatling sequences the cell ALU (owner ruling 2026-10-19): in the tetrad-integrated tile, gatling (the barrel-core tetrad engine) drives the cell's alu-infer ALU through `Xcell` instructions: `POP` presents operand line(s) to the ALU issue handshake, `PUSH` retires result line(s) into the result memory, and `FLUSH` programs `FLUSH_DEST`/`FLUSH_TAG` and triggers the drain. The slice-3 `cell_exec_fsm` remains the standalone bring-up driver with the same POP/issue → PUSH/retire client contract. Gatling never originates cell config traffic; cell programming stays in-band per the 2026-10-17 owner ruling. The `Xcell` encodings (`CFG`, `POP`, `PUSH`, `ECTX`, `FLUSH`, `KICK`, `WAIT`) live in gatling PRD v0.2.

v1 function set: eight opcode rows (copy, add, sub, mul, mac, min/max, compare/threshold, relu) defined with their `math_bf16_*` implementation base in alu-infer PRD §3; not restated here.

Opcode space reserves 5 bits; 8–31 are growth. Exact datapath widths and latency land in the MAS, consistent with the math area's learning ethos (sophisticated low-latency techniques).

2.5 5. Packet interface

The cell consumes and produces the §11.5 header unchanged; §11.6 locates it per transport. Fields the cell uses directly:

Field	Cell use
Destination coordinate	demux match against `{level=0, own X/Y/Z}`; rapids-bound emits use `FLUSH_DEST` (gatling-programmed). `DEST` is reserved for the deferred cell→cell path (TILE_SPEC §10).
Source coordinate	the tile header's source field (the originator — TILE_SPEC v1.6 §11.5 note; not delta's hop-source route header), recorded per arriving packet — credit-return destination and the deferred chain-back default for `DEST` (rapids sends as `level` 3, §11.3)
`ptype`	`mem` → input memory; `cfg` → registers; `credit` → credit/grant handler
`op`	write vs `cfg` read; credit `add` vs `set`
Length	payload bytes / credit count / read size
Target address	input-memory line index or register offset
Tag	echoed on replies and result packets so rapids routes them to the right sink channel

2.6 6. Credit behavior

2.6.1 6.1 Boot advertisement

After reset the cell sends one `set` credit packet (`op` = 1) to `level` 3 carrying its input memory's size **in flits**. Rapids's per-destination counter for this cell starts at 0 and cannot send until the advertisement arrives — the advertisement is the cell's "alive" (§11.10). The packet rides the credit VC and needs no grant.

2.6.2 6.2 Consumed-line credit returns

- Each input line consumed by the FSM owes **its sender** one credit return. The sender is the source coordinate recorded when the demux wrote the line (every input-memory line carries a source tag). Rapids (`level` 3) is the sender in the v1 flow, but the return is sender-generic per §11.9 — a credit return goes to the source coordinate of the packet whose flits it frees, so a cell that streamed into this cell's input memory receives the credits.
- Returns are `add` credit packets addressed to the recorded source coordinate, per §11.9(4): routed like any packet by destination coordinate; the sink port accepts them unconditionally and never stores them. They ride the credit VC and need no grant.
- **Batching (owner decision 2026-10-16, extended sender-generic 2026-10-18):** the credit logic keeps a per-source owed count and sends one credit packet per N consumed lines **per source**, with a timeout so a partial batch drains. **N = 1 is the default and the exact per-line behavior.** N > 1 trades return latency for less credit-packet traffic on the fabric.

2.6.3 6.3 Straps on the result path

The result path adopts §11.9's strap mechanism (owner direction 2026-10-18: credits toward RAPIDS are **straps, not grants**):

1. Toward RAPIDS the cell keeps no grant protocol. Space in RAPIDS's sink is fixed by construction: a per-cell strap of `TILE_LAND_FLITS` flits (a Hive-set parameter passed to RAPIDS, default 16 — TILE_SPEC §11.9 Hive contract), reserved at configuration.
2. The packetizer sends a result packet only while its strap counter covers the whole packet, header included, and spends the counter as it sends. RAPIDS returns `credit`/`add` as it drains the cell's landing space, restoring the counter.
3. Resync/re-baselining of the strap state follows RAPIDS's advertisement/resync rules (TILE_SPEC §11.10); exact timing is a tile §10 open item.
4. The cell never asks for result space — no ask path; the strap is the contract.

Result destination (owner direction 2026-10-18, TILE_SPEC §11.14): gatling hands the cell the drain destination. Result flushes go to `{ext 0, level 3}` (the tile I/O), source = this cell's external ID, tag = the sink channel gatling programmed. **The cell does not choose the destination.**

- **Operational drain:** `FLUSH_DEST` (coordinate) and `FLUSH_TAG` (sink channel) are programmed by gatling and used by the packetizer when `CTRL.flush-and-send` fires.
- **Reserved/deferred:** the legacy `DEST` register (offset `0x38`) is retained only for the future cell→cell result path. It is **not** used for rapids-bound drains. `DEST` becomes operational only after the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10); until then, v1 result traffic is RAPIDS-bound and `FLUSH_DEST`/`FLUSH_TAG` take precedence.

2.7 7. Config registers

AXI4-Lite, registers-only (house rule); routing/mode config travels in-band as `cfg` packets per the standing owner decisions — every register below is writable both from AXIL and from the `cfg` stream, and readable over AXIL for debug. v1 map:

Offset	Register	Notes
0x00	`CTRL`	enable, soft reset (also clears `IP`/`OP`), flush-and-send (packetizer)
0x04	`STATUS`	busy, input-empty, result-nonempty, strap counter nonzero (slice-1 RTL alias `status_grant_nonzero`)
0x08	`FUNC_SEL`	opcode (5b) + sub-op (2b) + reserved (2b) — element width is reserved under the bf16 model (alu-infer PRD §3)
0x0C	`OP_LINES`	input lines per function invocation
0x10	`EMIT_BATCH_X`	flits per result packet (packetizer)
0x14	`CREDIT_BATCH_N`	consumed lines per credit packet **per source**, default 1
0x18	`IRQ_EN`	done, error, strap-

Offset	Register	Notes
		starved (slice-1 RTL alias `IRQ_GRANT_STARVED_BIT`), credit-timeout enables
0x1C	`IRQ_STATUS`	write-1-clear status
0x20	`A`	persistent operand register A — 32b (2×bf16 lanes); feeds `op_line_a/b` per `OP_SRC`
0x24	`B`	persistent operand register B
0x28	`C`	persistent operand register C
0x2C	`D`	persistent operand register D
0x30	`T`	bf16 threshold immediate backing register (opcode 6)
0x34	`OP_SRC`	operand-source mux: `sel_a[2:0]`, `sel_b[2:0]` — 0 = input memory (pop), 1 = A, 2 = B, 3 = C, 4 = D; reset 0 (both operands from memory)
0x38	`DEST`	*reserved for the deferred cell→cell path (2026-10-18/19 ruling)* — the rapids-bound drain destination is gatling-programmed into `FLUSH_DEST`/`FLUSH_TAG`; the cell does not choose it. `DEST` is retained (bits [13:0] = §11.3 coordinate at `COORD_W` = 4; bit [16] = override) but is **not** used

Offset	Register	Notes
		until the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10).
0x3C	`IP`	input-memory read pointer — auto-increments on pop, wraps at depth
0x40	`OP`	result-memory write pointer — auto-increments on push, wraps at depth
0x44+	counters	lines in, lines out, credits returned, packets sent (§8)

The `A–D/T` window is written by `cfg` packets like any other register (§3); loading `A–D` *from* the input memory is deferred (§11).

2.8 8. Status and observability

- IRQs: computation done, error, strap-starved (a packetizer with data and no strap, time-limited), credit-timeout.
- Software counters per §7.
- One MonBus observer stream per cell, on AXI4-Lite (owner direction 2026-10-19), per the tile's §5 aggregation hierarchy: the cell's stream is a leaf the tetrad-level merge consumes, alongside the other three cells and the tetrad node. Exact leaf selection is the tile §10 open item.

2.9 9. Verification hooks

cocotb + rds-dv BFMs per the standing convention. Test levels follow the house gate/func/full grid. Natural hooks: drive the node from a GAXI master instead of a mesh (the node's local port is a normal stream face), force credits and grants from the TB, and check the credit loop (count lines in → credits out, per recorded source) independently of the datapath.

2.10 10. Dependencies

- delta — the backbone network — **in progress**, built by a separate agent (fabric-gen-ip/delta). The cell instantiates one `delta_mesh_cell` node as a black box; modes 9–10 diagonal ports landed (Task 6c), the rest of the backbone is being built out.
- rapids — the DMA running data in and out of the tile — **in progress**, separate lane. The cell's dependencies on it are the per-destination credit counters, strap pool (`TILE_LAND_FLITS`), and sink-to-source credit-return path (§11.9 requirements list); sequencing with the rapids lane.
- rapids_cxs + delta CXS migration — **in flight** (amba lane); the cell's §11.6 transport note applies unchanged until delta ports move.
- TILE_SPEC §10 open items that touch the cell: input-buffer sizing, grant allocation policy, resync timing, cell→cell grant mediation (needed for cell-destined `DEST`, §6.3).
- the alu-infer PRD — the ALU block's product definition (bf16 element model, function set, interface contract, control interface); **normative for the ALU** since 2026-10-17.
- the alu-infer MAS — the implementation-intent book (module tree, function rows, accumulator, interface, cycle behavior, verification, configuration); lives in this directory's `docs/alu_infer_mas/`.
- a tetrad-level RV32I control core per alu-infer PRD §5 — **deferred** (owner direction 2026-10-18): the tetrad engine is defined once the cell is fully defined and working, with the owner-authored `gatling-rv32itetrad` barrel-core PRD the leading candidate and kestrel-class a candidate flavor via `CORE_FLAVOR`. When it lands, gatling sequences the cell ALU through `POP`/`PUSH`/`FLUSH` (owner ruling 2026-10-19); the cells see its local AXIL register plane and its operational writes only. Cell *programming* stays in-band (owner ruling 2026-10-17).

2.11 11. Deferred

- CGRA array of richer PEs (2026-10-13 ambition): the cell's block boundaries — node, input memory, compute, result memory, packetizer — are chosen so an array replaces the ALU+FSM block without disturbing the network, credit, or observation contracts.
- Context switching / multi-function pipelines.
- Cell-initiated grant requests.

- Wider-than-32-bit lines (element widths stay bf16-only regardless; §4).
- Loading `A`–`D` from the input memory (v1 writes them via `cfg`/AXIL).
- Cell-destined result sending (`DEST` = another cell) pending the tile-spec cell→cell grant extension (§6.3).
- The tetrad-level control core and the gatling/tetrad engine (§10).

2.12 12. Open questions

- Input memory and result memory depths (lines, and flit-conversion for the advertisement) — sized in the MAS against the MAS injection-FIFO and credit-flow analysis.
- `cfg` read reply path: whether rapids's sink half strips headers (TILE_SPEC §11.7 open item) affects nothing here — the cell always builds a full reply header.
- Flush conditions for the packetizer beyond batch-X (idle timeout? end-of-job marker from the FSM?).
- Cell→cell grant/advertisement rules: who grants sink space for cell-destined results, and how a cell advertises its input-memory credits to peer cells (today: to rapids only, §11.10). Needed for `DEST` overrides (§6.3); lands in the tile spec first.
- `DEST` coordinate width tracks the §11.3 `COORD_W` parameterization (4 today); the register is 32-bit with the used low bits defined above.

2.13 References

- `alu-infer/PRD.md` — the ALU block's product definition (normative for the ALU since 2026-10-17).
- `tile/docs/TILE_SPEC.md` — adopted tile hierarchy, §11 packet header, credits/grants (normative for §§3, 5, 6).
- `delta_has/ch07_mesh_network/` — the node the cell instantiates.
- `riscv-ip/gatling-rv32itetrad/PRD.md` — owner-authored barrel-core tetrad-engine PRD (leading candidate; deferred per §10).
- `axon_spec.md` — superseded 2026-10-16 (decision trail).
- `2026-10-13-delta-mesh-design.md` — working design record.

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

← Back to tile

3 Tile — Compute Engine Tile Subsystem Specification

Version: 1.8 (2026-10-19; §11.13/§11.14 tetrad control core and result flush — owner direction: gatling sequences the four alu-infer ALUs of its adjacent cells via `POP`/`PUSH` against the cell ALU issue/result contract, the cell slice-3 exec FSM stays the standalone bring-up driver with the same client contract, and gatling never originates config traffic; `Xcell FLUSH` programs the cell's `FLUSH_DEST`/`FLUSH_TAG` and triggers flush-and-send; §12 milestone ladder M1–M4 added). Carries 1.7 (2026-10-19; §5 MonBus per level — owner direction: every cell emits one MonBus on AXI4-Lite, the tetrad merges every MonBus at its level (4 cells + the tetrad node) into one AXI4-Lite out, and the tile merges the 4 tetrads and the tile-level nodes (root 0, root 1) into one MonBus). Carries 1.6 (2026-10-18; §11.5–11.6 header-transport binding — two-header model: delta route header in `TUSER` (AXIS), the 128-bit tile header as the first data flit on both transports per the RAPIDS built implementation; §11.2 cell-traffic QoS rule — QoS 0 reserved for the node's own config plane, all cell-bound traffic rides QoS ≠0; `TILE_LAND_FLITS` home is the Hive top. Clarifications, no mechanism change. Carries 1.5 (2026-10-18; §11.13 tetrad control core **deferred** pending the cell — owner direction 2026-10-18, gatling-rv32itetrad the leading candidate; cell→cell grant mediation added to §10 open items; §11.3 internal/external ID on every node, §11.9-11.10 credits — owner direction 2026-10-18: delta nodes are conduits for transit and endpoints for what they consume, 32 RAPIDS credit counters, the advertisement window's saved legal-destination mask, and a cell's advertisement sent by its delta node from a parameter, then (same day) delta nodes only transport and never originate: a cell or core advertises for itself, nodes are legal by topology, and straps toward RAPIDS replace grants; §11.14 result flush to the tile I/O, delta must carry result data to `level` 3). Carries 1.4 (tetrad control core recorded 2026-10-17; §11 adopted 2026-10-16; cell integration decisions 2026-10-16; §11 proposal added 2026-10-16; re-homed to compute-eng-ip 2026-10-15; v1.0 owner-defined 2026-10-15) **Status:** Canonical microarchitecture definition for the compute-engine tile. **delta is the node** (`delta_mesh_cell`) — nodes collectively form the network; this document defines the tile IP that instantiates 16 of them. Where this document and `delta_has/ch07_mesh_network/01_hierarchy.md` disagree, this document wins — the hierarchy chapter predates the 2026-10-15 owner decisions (tetrad/tile naming, real wrapper modules, MonBus aggregation, this re-home). The working design record with full citations is `docs/superpowers/specs/2026-10-13-delta-mesh-design.md` (§2 in particular); this spec is its distilled, decision-frozen form.

3.1 1. Scope

The **tile** is the delta mesh's subsystem unit: 16 cells arranged as 2×2×1 **tetrads** of 2×2×1 cells, fully connected at both scales, with per-level MonBus observation aggregation and a single external attach point for rapids. This document defines the tile's hierarchy, modules, connectivity, observation architecture, interfaces, and mode set.

Out of scope: cell-internal microarchitecture (buffering, allocators, VCs — design spec §3 and ch07 §§4–6), flit format (§4), the flat crossbar modes of the delta generator's other configurations, and the fabric-level Z stacking above one tile.

3.2 2. Hierarchy

Level	Composition	RTL	Purpose
Cell (node)	1	`delta_mesh_cell` — owned by `fabric-gen-ip/delta`	the one module with datapath: routing tables, VCs, allocators
Tetrad	2×2×1 cells (4)	`tetrad` (this IP)	datapath grouping (K4) + first MonBus aggregation
Tile	2×2×1 tetrads = 4×4×1 cells (16)	`tile_top` (this IP)	the subsystem: full K4 at tetrad scale, tile-top MonBus, rapids attach
Fabric	4×4×K tiles	generated top	planar recursion; Z lives between tile layers

Owner-named 2026-10-15: “tetrad” (was “tile”) and “tile” (was “confluence”).

Adopted 2026-10-16: §11 refines this hierarchy — the tetrad is a routing node with datapath (not a passive wrapper), two root nodes sit above the tetrads behind a root mux, and rapids attaches at the tile I/O above the mux (§11.1, §8 below). The table’s RTL column names predate that decision; the wrapper-shaped `tetrad`/`tile_top` plan built on it is superseded. The hierarchy’s leaves are the cells, defined by the cell PRD (`compute-eng-ip/cell/docs/PRD.md`) — each a self-contained IP whose only tie to delta is one node instantiated inside it.

Owner direction 2026-10-17: each tetrad additionally carries **one RV32I control core** (kestrel-class, `CORE_FLAVOR`-swappable) at its tetrad node’s local port — the control agent for the tetrad’s four cells, defined with its duties and boundaries in alu-infer PRD §5. All data, config, and programming for the tile blocks — the core’s firmware included — arrive in-band on delta; the core is a local reaction agent, not a config originator. SERV is unchanged in its Ocelus observability role. See §11.13.

3.3 3. Connectivity

Connectivity is the **same K4 clique at both scales** — the owner’s figure `assets/3d-mesh-tile.drawio` draws it: four fully meshed cells inside the tetrad, four fully meshed tetrads inside the tile.

- **Cell ports**: 6 neighbors (±X, ±Y, ±Z) + 1 local (host/rapids-facing)
  - P_DIA + P_DIA2 + 1 MonBus observer out.
- **Intra-tetrad K4**: the 4 perimeter edges ride ordinary ±X/±Y links; both diagonals ride **P_DIA**, one link per cell to the diagonally opposite cell of its tetrad.
- **Intra-tile K4**: the 4 tetrads of a tile are themselves fully connected. Edge-adjacent tetrads share ±X/±Y boundaries; diagonal tetrad pairs ride **P_DIA2**, joining the diagonally adjacent corner cells (e.g. (1,1,0)↔(2,2,0), (1,2,0)↔(2,1,0) per tile). Out-of-grid mates tie P_DIA2 off — provably never selected.
- **One conceptual diagonal axis**: P_DIA and P_DIA2 are two instances of a single diagonal axis realized at tetrad and tile scale. A cell has 4 axes in-plane-plus-diagonal plus Z: 8 directional links + local = 9 ports.
- **Z**: entirely inter-tile; tetrads are planar (X/Y only).

3.4 4. Modules

- `delta_mesh_cell` — the node, owned and emitted by delta (the fabric-gen-ip component). The only datapath module: routing tables, VCs, allocators. The network is the collective of instantiated nodes; this IP is not it.
- `tetrad` (`rtl/tetrad.sv`, this IP) — instantiates 4 delta nodes; wires the K4 connectivity of §3; merges every MonBus stream at its level (the 4 cells and the tetrad node, §11.1) into one tetrad MonBus on AXI4-Lite (§5). No datapath function.
- `tile_top` (`rtl/tile_top.sv`, this IP — house `<ip>_top` naming) — instantiates 4 tetrads; merges the 4 tetrad MonBus streams and the tile-level nodes' streams (root 0, root 1) into the single tile-top MonBus (§5); exposes boundary nodes' local ports for rapids attachment. No datapath function. (Working names during the delta-side design phase were `delta_mesh_tetrad` / `delta_mesh_tile`; retired with this re-home.)

Adopted 2026-10-16 (§11.1): the tetrad becomes a routing node with datapath, so its module owns switching above its group of 4 cells; the tile adds 2 root nodes, a root mux, and the single tile I/O. The wrapper-only description above is kept for the decision trail; module boundaries for the adopted shape are part of the tile RTL planning and follow the cell PRD's block structure.

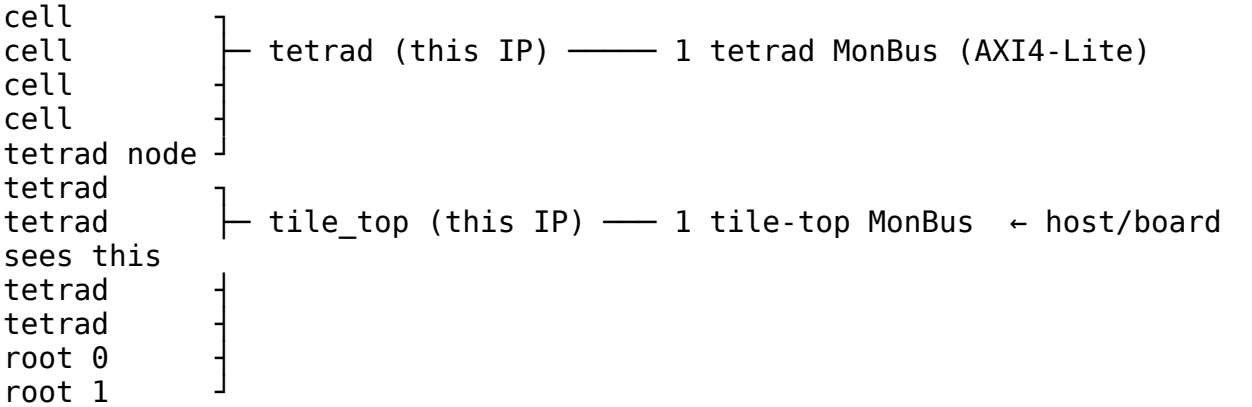
Owner direction 2026-10-17: the tetrad module also instantiates the tetrad's **control core** — one kestrel-class RV32I core (this IP) attached at the tetrad node's local port, per §11.13 and alu-infer PRD §5.

3.5 5. Observation (MonBus aggregation)

Owner decision 2026-10-15; refined by owner direction 2026-10-19. One MonBus leaves each level, and each level merges **every** MonBus source at that level, nodes included, not only the level below:

MonBus sources per level

Level	MonBus sources merged	Out
Cell	the cell	one MonBus, on AXI4-Lite
Tetrad	its 4 cells + the tetrad node (+ the control core, §11.13, when it exists)	one MonBus, on AXI4-Lite
Tile	the 4 tetrads + the tile-level nodes: root 0, root 1 (the root mux is not a node, §11.1)	one MonBus



Aggregation uses the house AXIL MonBus group family (`rtl/amba/ocelus/monbus_group_core` + `monbus_*_group` wrappers, AXIL host-read flavor) — the same observation path as the amber tally harness (merge → group → AXIL read-out), so standard host tooling (`monbus_tally_axil`, regmaps) attaches at the tile top unchanged. Exact leaf selection inside the wrappers is implementation detail; the architectural contract is **one MonBus per level boundary, merging every source at that level**: 5:1 at the tetrad (6:1 once the control core exists), 6:1 at the tile.

This hierarchy is also the verification anchor: per-tetrad and per-tile observability come for free, and the tile-top stream is the single sim-vs-silicon equivalence point (the same role the UART-equivalence cell plays for the amber harness).

3.6 6. Interfaces

- **Cell/tetrad/tile datapath ports**: AXI4-Stream (`TDATA/TVALID/TREADY/TLAST` + header sideband in `TUSER`), with the optional AMBA 5 TSTRB/TWAKEUP/TPARITY richness carrying over from flat mode unchanged.
- **CXS migration (owner plan, 2026-10-15)**: rapids gains a CXS interface variant — a separate model, `rapids_cxs`, replacing its AXIS interfaces — and delta's network ports move to CXS to match. Sequencing: rapids_cxs first (a working AXIS model exists to diff against), then delta. Header/sideband content is unchanged by the migration. Until both sides land, all delta ports stay AXI4-Stream.
- **MonBus ports**: per §5; cell-level streams are internal to the tetrad, tetrad-level streams internal to the tile.

3.7 7. Modes

The mode catalog (design spec §5.4) is unchanged by this spec; the tile-relevant rows:

Mode	Pattern	Tile relevance
1 Mesh (default)	DOR over RAM tables	baseline; inter-tile dsts DOR
9 Tetrad all-to-all (K4)	perimeter ±X/±Y + P_DIA diagonals	intra-tetrad 1 hop to any cell; inter-tetrad DOR
10 Tile all-to-all (K4 of tetrads)	shared boundaries + P_DIA2 diagonals	any cell of any tetrad of the tile in 1–2 hops (diagonal-of-tetrad pairs 1 hop); inter-tile DOR

Modes are table images, not new hardware — except that modes 9–10 make P_DIA/P_DIA2 (emitted in every build) their enabling hardware.

3.8 8. Rapids attachment

Hive configuration (owner, 2026-10-15): one or more tiles — only one for now — connect to a **single** rapids DMA at boundary cells' local ports; any boundary cell can be the attachment. With the CXS migration (§6), this link is the first CXS hop.

Adopted 2026-10-16 (§11.1): rapids attaches above the two root nodes through the root mux, at the single tile I/O — not at a boundary cell's local port. Its descriptors, buffers and channel state

survive a root failover (§11.4); the cell↔rapids protocols (packet header, end-to-end credits, straps) are §§11.5–11.10.

Status (owner, 2026-10-18): rapids is the DMA running data in and out of the tile; RTL **in progress** on a separate lane, alongside the delta backbone (an agent is building that).

3.9 9. Clocking and reset

Single clock domain, active-low async `aresetn` (house `ALWAYS_FF_RST`) throughout the tile — a deliberate simplification against the NoC literature's GALS assumption; timing closure and generator predictability win over clock flexibility (design spec §2 close).

Resets and partial resets (a tile, a tetrad, RAPIDS or one of its channels) happen only during quiescent periods, under the global quiescent signal (§11.9, hazard rules). How each partial reset sequences against the credit state is still to be worked out (§10).

3.10 10. Open items

Decided 2026-10-16 (was §11): the hierarchical tile of §11.1 (tetrad routing nodes, two roots, root mux, tile I/O); the packet header and payload types (§11.2, §11.5); the destination coordinate (§11.3); end-to-end credits with boot advertisement, per-destination counters, and straps gating cell→rapids traffic (§§11.9–11.10, owner direction 2026-10-18, replacing grants); credit-return batching as a per-cell config with default N=1 (cell PRD §6).

Still open (see the milestone ladder in §12):

- The global quiescent signal (§11.9): how it is formed across every cell and tile, and how each reset and partial reset sequences against it.
- What switches the root mux (§11.4): software, a watchdog, or the root-to-root link.
- Cell input buffer sizes (the advertisement value) and result-memory sizing — cell PRD §12.
- Resync timing: how long RAPIDS waits for answers before declaring a destination dead (§11.10).
- Cell-issued descriptors (§11.11): whether a source channel is reserved for them or any channel may take them, and whether a cell is told when its pull completes.
- CXS leaf selection for both rapids_cxs and delta ports (§6).
- MonBus wrapper leaf selection inside tetrad/tile (§5).
- Config/address region naming per level — design spec §5, ch07 §3; §11.3's coordinate is the addressing answer, register naming follows the cell PRD.

- Cell→cell flow control (§11.9): credit *returns* are sender-generic already — a credit goes to the source coordinate of the packet whose flits it frees — but straps are defined only toward RAPIDS. Who reserves space for cell-destined results, and how a cell advertises its input-buffer credits to peer cells, is undefined. Needed for the cell PRD's `DEST` override (cell PRD v0.3 §6.3); v1 result traffic is rapids-bound.
- Result flush (§11.14): the gatling-to-cell command is now defined as the `Xcell FLUSH` custom instruction (gatling PRD v0.2); the cell's flush-and-send CSR details and encoding land in the cell/tetrad MAS.
- Tetrad control core (§11.13): the 2026-10-19 ruling records gatling sequencing the four cell ALUs and no config origination; the `KICK`/`WAIT`/`FLUSH` encoding, local memory map, and program-load conventions remain cell/tetrad MAS work. **Deferred** (owner direction 2026-10-18): the engine is defined after the cell is fully defined and working; gatling-rv32itetrad is the leading candidate (§11.13).
- Multi-path routing and failover (§11.15, delta DELTA-013): fully adaptive or pinned forward with an adaptive return; the transaction ID that makes ordering a requester-side property.

Tile RTL (`tetrad` node, `tile_top` with roots + mux + I/O) is pending the adopted-shape module split above; the node RTL (`delta_mesh_cell`, modes 9–10 diagonal ports, Task 6c) has landed in `fabric-gen-ip/delta`.

3.11 11. Hierarchical tile, root nodes, addressing (ADOPTED 2026-10-16)

Status: ADOPTED, owner decision 2026-10-16. Where §§2–4 and §8 still carry the pre-adoption wrapper wording, this section is the decision record and wins. Residual open sub-items are listed in §10.

3.11.1 11.1 Hierarchy

Level	Count per tile	Role
Cell	16, in 4 groups of 4	Leaf node; the 4 cells of a group are fully meshed (K4)
Tetrad node	4	A routing node above its group of 4 cells; the 4 tetrad nodes are fully meshed (K4)

Level	Count per tile	Role
Root node	2 (root 0, root 1)	A routing node above the tetrads; each root links to all 4 tetrad nodes; the two roots are linked to each other
Root mux	1	Selects the live root; not a node and not addressable
Tile I/O	1	The single connection out of the mux, to the one RAPIDS: its **source port** (RAPIDS to tile) and its **sink port** (tile to RAPIDS)

Changes against §§2-4: - The tetrad becomes a routing node with datapath, not a wrapper. - RAPIDS attaches above the roots, through the mux, not at a boundary cell's local port.

3.11.2 11.2 Payload types

Every packet carries a payload type, chosen by the RAPIDS descriptor that produced it:

Code	Type	Target
0	`cfg`	Node or cell configuration (routing tables, mode, CSRs)
1	`prog`	Program load
2	`mem`	Node or cell memory
3	`credit`	End-to-end credit advertisement or return (§11.9); header only
4	`desc`	A cell-issued RAPIDS source descriptor (§11.11)
5-7	reserved	-

Delta QoS classes (owner clarification 2026-10-18): QoS 0 is **reserved for the delta node's own config plane** — a node consumes any head flit with `dst == POS && qos == 0` as route-table

config and never ejects it to its local port. All cell-bound traffic (`cfg`, `prog`, `mem`, `credit`, `desc`) therefore rides delta QoS ≠ 0: class 1 (latency) for `cfg` / `credit` / `desc`, class 2 (bulk) for `mem` / `prog`. A cell's demux dispatches by the tile header's `ptype` (§11.5), never by QoS.

3.11.3 11.3 Destination coordinate

Proposed destination coordinate, `{ext, level, X, Y, Z}`, 15 bits at COORD_W = 4

Field	Width	Meaning
`ext`	1	Which of the node's two IDs (owner direction 2026-10-18): 0 = internal, the node itself; 1 = external, what is attached at its local port
`level`	2	0 = cell-level node, 1 = tetrad node, 2 = root node, 3 = tile I/O (RAPIDS)
X	`COORD_W` (4)	Cell: its X. Tetrad: its group's X (the cell X with the low bit dropped). Root: root 0 or 1. Tile I/O: reserved.
Y	`COORD_W` (4)	Cell: its Y. Tetrad: its group's Y. Root and tile I/O: reserved.
Z	`COORD_W` (4)	Tile layer, unchanged from delta

- **Every node has an internal and an external ID** (owner direction 2026-10-18). The internal ID (`ext` = 0) is the node itself: its routing tables, its configuration, the buffer for packets addressed to it. The external ID (`ext` = 1) is what is attached at its local port: at a cell-level node, the cell; at a tetrad node, its control core (§11.13).
- A cell is therefore the external ID of its cell-level node, `{1, 0, X, Y, Z}`. Tetrad nodes, roots and the tile I/O are addressed by their own level, never as a cell.

- Only a node with something attached has external credit (§11.9): an external ID with nothing behind it never advertises, so it is never a legal destination.
- Routing uses `{level, X, Y, Z}` only; `ext` is read at the last hop, to consume the packet in the node or deliver it out of the local port.
- The mux has no coordinate.
- A packet's source coordinate uses the same format. RAPIDS always sends as `level = 3`, the tile I/O (`ext` = 0), so replies reach it through whichever root is live.
- Delta's head-flit header carries `{X, Y, Z}` today (ch07 §2). Adding `level` and `ext` is a delta header change, still to be made. It is required: delta must deliver result data to `level` 3 (§11.14).

3.11.4 11.4 Root failover

- Root 1 takes over if root 0 fails. There is one RAPIDS, so its descriptors, buffers and channel state survive a root failure.
- Packets in flight inside the failed root are lost. RAPIDS sees those transfers stall and time out (its scheduler timeout), and software re-issues them.
- The mux switches only at a packet boundary. On the CXS link to RAPIDS, a switch is a link deactivation followed by a reactivation, which the CXS endpoints already handle (credits are returned before deactivation completes).
- Open: what drives the mux select (§10).

3.11.5 11.5 Packet header

Every packet starts with one header. It extends delta's head-flit header (delta HAS ch07 §2: coordinates, QoS class, VC, multicast, misroute budget, source-route pointer, packet ID) with the fields below.

Proposed tile packet header fields (beyond delta's existing ones)

Field	Width	Meaning
Destination coordinate	15	`{ext, level, X, Y, Z}`, §11.3; adds `ext` and `level` to delta's `{X, Y, Z}`
Source coordinate	15	Same format; RAPIDS sends as `level` = 3, the

Field	Width	Meaning
		tile I/O
`ptype`	3	Payload type, §11.2
`op`	1	Data packets: 0 = write (payload follows), 1 = read (header only; the target replies). `credit` packets: 0 = add, 1 = set (§11.10)
Length	32	Write: payload bytes that follow. Read: bytes requested. Credit: flits (or descriptor entries) advertised or returned. `desc`: 32 (one descriptor).
Target address	49	Byte address inside the addressed node or cell. Credit packets: bits [1:0] are the credit class (0 = sink space, 1 = descriptor slots), the rest reserved.
Tag	4	The RAPIDS channel. Set by RAPIDS on a request; echoed by the target in its reply, so the reply reaches the right sink channel

Proposed packing, for a transport that carries the header as data:

Proposed header packing

Bits	Field
`[14:0]`	Destination coordinate
`[29:15]`	Source coordinate
`[32:30]`	`ptype`
`[33]`	`op`

Bits	Field
[65:34]	Length
[114:66]	Target address
[118:115]	Tag
above [118]	Delta's existing head-flit fields, then reserved

Two headers, two owners (owner clarification 2026-10-18): this tile header is a **payload** header — the first data flit of the packet. It is distinct from delta's **route header**, which delta owns and which rides TUSER of the head beat on AXI4-Stream (§11.6). Consumers that need the packet's *originator* — credit returns (§11.9), a cell's recorded source — read the tile header's source coordinate; delta's route-header `src` is the previous hop, not the originator.

The header costs one flit per packet. For short `cfg` writes that is a large share; a later revision could pack a small payload into the reserved bits.

3.11.6 11.6 Transport

The header is the same whatever carries it — a **data flit**, per the RAPIDS built implementation (`rapids_pkg::tile_hdr_pack`, 128 bits; see §11.12). Delta's route header (§11.5 note) is separate and stays delta's:

Transport	Delta route header	Tile packet header (§11.5)
AXI4-Stream (delta ports today)	TUSER of the head beat, as delta does now	First data beat of the packet (`tdata[127:0]` of beat 0)
CXS (RAPIDS CXS tops; delta ports after the migration in §6)	Per delta's CXS port mapping (delta migration, §6)	The data of the first flit of the packet

Binding (owner clarification 2026-10-18): the earlier "AXIS: TUSER of the head beat" wording for the *tile* header is superseded — on AXIS, TUSER carries only delta's route header. Until delta ports move to CXS, a cell on an AXIS local port sees: TUSER = delta route header, beat-0 `tdata` = tile header, then payload beats.

Beat-chained header (owner ruling 2026-10-18): every link keeps its own data width; the 128-bit tile header occupies the first `ceil(128/DW)` payload beats of the packet, LSB-first (on a 64-bit link: beat 0 = header `[63:0]`, beat 1 = header `[127:64]`). No link widens and no width converter is needed anywhere in the fabric for the header's sake; the tile I/O link (256-bit) carries header and payload together.

On CXS the link's own sidebands keep their link-level meaning: RAPIDS keeps the channel in CXSSRCID, so per-stream framing is unchanged.

3.11.7 11.7 Where the fields come from

- **RAPIDS:** its descriptor supplies `ptype`, `op`, the destination coordinate and the target address (RAPIDS HAS, Descriptor Format); the tag is the channel that issued the descriptor. The source half builds the header and sends it out of the source port, ahead of any payload.
- **Nodes:** none. A delta node never originates a packet (§11.9), so it never answers a read; a cell or core answering one builds a reply header addressed to the request's source coordinate.
- **Open:** whether the RAPIDS sink half strips the header or delivers it to memory.

3.11.8 11.8 Operations

Only cells and control cores send into RAPIDS's sink port, and only against their straps (§11.9); a delta node never does.

Proposed operations between RAPIDS and the tile

Operation	Source port sends	Target does	Sink port receives
Write	Header (`op` = 0, length N, tag) + N bytes	Consumes the data, then returns credits (§11.9)	The credit packet
Read	Header only (`op` = 1, length N, tag = the sink channel that will take the data); never to a node	Sends N bytes back: a write-format packet to `level` 3 with the tag echoed	The data, into the tagged sink channel
Cell send	-	Sends a write-format packet to `level` 3 with the tag set, against its `TILE_LAND_FLITS` strap	The data, via the cell's landing space, into the tagged sink channel
Credit return to a sender	Header only (`ptype` = `credit`, destination = the cell or core) for what RAPIDS	Adds it to its strap counter	-

Operation	Source port sends	Target does	Sink port receives
	consumed		
Credit return	-	-	Header only (`ptype` = `credit`) for flits the cell or core consumed; never stored in a channel
Core-issued descriptor	-	Sends a `desc` packet to `level` 3: header + one 256-bit source descriptor, tag = the source channel, against its strapped descriptor-queue entries	The descriptor, into that source channel's descriptor queue (§11.11)

A read carries its own reservation: RAPIDS issues it only when the tagged sink channel has room for N bytes, and the reply needs no strap credit.

3.11.9 11.9 End-to-end credits

Link credits (CXS `CRDGNT` / `CRDRTN`) cover one hop and carry no address, so they cannot say anything about a cell's buffer. End-to-end credits are packets instead.

delta nodes only transport (owner direction 2026-10-18). A node carries every packet on the data network, and `credit` packets on their own interface, and holds no credit state for it: only the hop-level flow control it already has. A node never originates a packet on the data network; whatever it generates itself (debug, status) goes on MonBus. It consumes what is addressed to its internal ID (§11.3) on arrival, so it needs no end-to-end credit: it never advertises and never returns credits, and it must never stall what is addressed to it. End-to-end credits belong to what is attached at a node's local port, under the node's external ID: a cell, or a tetrad's control core (gatling).

1. RAPIDS's source half keeps **64 credit counters** (owner direction 2026-10-18), one per destination index `{ext, n}`: the internal and the external ID of every node. A tile uses at most 42 (22 internal: 16 cell-level nodes, 4 tetrad nodes, 2 root nodes; 20 external: 16 cells, 4 tetrad control cores). An external counter starts at 0 and is filled by its cell's or core's own advertisement (§11.10), so an external ID with nothing attached is never legal. An internal (node) index is legal by topology: a packet to a node is never held for credits and never charged.

Index	Destination
0-15	Cell-level node (internal), `{Y[1:0], X[1:0]}`
16-19	Tetrad node (internal), `{Y[0], X[0]}` of its group
20-21	Root node 0, 1 (internal)
22-31	Spare
32-47	Cell: the cell-level node's external ID, `32 + {Y[1:0], X[1:0]}`
48-51	Tetrad control core: the tetrad node's external ID
52-53	Root external: none (the root's local side is the tile I/O)
54-63	Spare

: Proposed credit-counter index

2. Each flit sent to a destination, the header included, spends one credit. RAPIDS sends a packet only when the counter covers all of it.

3. The cell or core returns credits when it has **consumed** the flits (written them to memory, or applied the `cfg`), not when they arrive. It may batch them, for example one return per packet.

4. The return is a `credit` packet to `level` 3 with the count in the length field. The sink port recognises it, never stores it, and passes {source coordinate, count} to the source half, which adds the count to that destination's counter.

Counters are per destination, not per channel: two channels writing the same cell share that cell's buffer. Traffic routed through a node never touches a counter. A node never replies, so a read request to a node's internal ID is refused as an invalid descriptor; node registers are read over MonBus or AXIL.

Where a credit goes: a credit packet is routed like any packet, by its destination coordinate. A credit return goes to the source coordinate of the packet whose flits it frees, so a cell returning credits to RAPIDS always addresses `level` 3, and a credit return from RAPIDS goes to the cell or core whose packet it consumed.

Credits toward RAPIDS: straps, not grants (owner direction 2026-10-18). A cell or core must never send into the sink port without guaranteed space: flits that cannot drain stall in the routers and block everything behind them, credit packets included. The space is fixed by construction instead of granted at run time:

Credits toward RAPIDS

Sender	Sends to RAPIDS	Credit
Cell (a cell-level node's external ID)	Memory writes (results, §11.14)	A strap of `TILE_LAND_FLITS` flits (a Hive-set strap passed to RAPIDS, default 16; §11.9 Hive contract below), header included
Control core (a tetrad node's external ID)	`desc` packets (§11.11)	Entries in RAPIDS's `desc` intake queue
delta node	Nothing	None

1. Each sender starts from its strap after reset and sends a packet only while its counter covers the whole packet.
2. RAPIDS reserves every strap: a landing space of `TILE_LAND_FLITS` flits per cell, and a `desc` intake queue as deep as the descriptor credits handed out. The sink cannot be overcommitted however many senders send.
3. RAPIDS returns credits as it consumes, with a `credit`/`add` packet to the sender: flits when the data moves out of the landing space into its sink channel, one entry when a descriptor moves from the intake queue into the descriptor queue. These returns are required, not optional.

The landing space is per sender: a cell whose data targets a full sink channel holds only its own landing space, never another sender's. A packet to RAPIDS is therefore at most `TILE_LAND_FLITS` flits.

With end-to-end credits in both directions, every packet in the fabric has guaranteed space at its destination and always drains.

Requirements this places on the implementation: - **Deadlock:** a credit must never wait behind data that is itself waiting for credits. Credits have their own interfaces (owner 2026-10-18), separate from data, so this holds by construction. - **RAPIDS:** the sink-to-source credit path inside the core, the 64-entry counter table (filled by the advertisements), the advertisement window and the saved legal-destination mask (§11.10); and toward RAPIDS, the `TILE_LAND_FLITS` landing space per cell, the `desc` intake queue, and the credit returns. - **Cells and control cores:** buffer sizing, their own boot advertisement, return batching, building credit packets, and a strap counter toward RAPIDS. - **delta nodes:** none of it. No credit state, no advertisement, no returns; they consume what is addressed to them without stalling. `credit` packets travel on their own interface.

Hazard rules (owner direction 2026-10-18): - **Always return credits.** Every packet RAPIDS consumes returns its full credit, including one it discards (bad header, bad tag, disabled channel).

No path consumes a packet silently. - **Stalled data is a monitor error, both directions.** RAPIDS carries ocelus checks, each a threshold in cycles (10 us) that flags an error on MonBus: - Sink: data present in the sink SRAM that has not moved. - Source: a source channel expecting data (reads outstanding) whose SRAM has received none. - **Return credits promptly.** RAPIDS may coalesce the returns already pending when it sends one, but never holds credits back to build a batch while the sender may be waiting for them. - **Straps are a Hive-level contract.** The Hive sets every strap and parameter (`TILE_LAND_FLITS`, the descriptor credits, the cells' and cores' straps) and checks they agree; it is the final arbiter. A block built on its own (one tetrad, one RAPIDS) is only as consistent as the parameters it is given. - **Resync and reset only when quiescent.** A global quiescent signal, covering every cell and tile, gates any resync and any reset or partial reset (§9, §11.10), so no packet is in flight against credits being restored or cleared.

3.11.10 11.10 Credit advertisement

Each side advertises its own credits in-band. Nothing is loaded from CSRs, so the counts cannot drift from the hardware.

Credit packet operations

op in a `credit` packet	Meaning	Used by
0 = add	Add N to the counter	Credit returns, both directions
1 = set	Replace the counter with N	Advertisements at boot, resync

Boot: 1. After reset every cell and control core sends one `set` credit packet to `level` 3, under its external ID, carrying its full input-buffer size in flits (owner direction 2026-10-18: the cell or core originates it; a delta node never does). RAPIDS's external counters start at 0, so RAPIDS cannot send to a cell or core that has not advertised. delta nodes do not advertise: they are legal by topology. 2. The advertisements ride delta's hardwired reset-default route toward the RAPIDS boundary (delta HAS ch07 §3), so no routing table is needed first. 3. They need no strap credit: credit packets travel on their own interface and the sink port always accepts them. 4. The advertisement doubles as the cell's or core's "alive": RAPIDS accepts strap traffic only from a sender that advertised in the window. 5. **Legal destinations** (owner direction 2026-10-18). RAPIDS opens an advertisement window when it leaves reset. When the window closes, every node's internal ID is legal, and every external counter still at 0 marks an **illegal** destination: nothing there advertised, whether an absent cell, an absent core or a spare index. RAPIDS saves the result as a 64-bit legal-destination mask, one bit per counter. - A descriptor whose destination is illegal is refused as an invalid descriptor, not sent and not left waiting for credits that will never come. - Only the window decides legality. A legal destination whose counter later reaches 0 because its credits are spent is waiting, not illegal. - The mask is readable by software (a CSR) and stays as saved until the next reset.

RAPIDS's side: straps, not advertisements. RAPIDS never advertises its free sink space as a total. If it broadcast "channel 3 has 512 flits free", every cell would believe it could send 512 flits,

and the sink would be overcommitted many times over. Its space toward each sender is a fixed strap instead (§11.9), reserved by construction.

Resync after root failover: credit packets in flight in the failed root are lost. Without a resync the lost credits leak until the counters reach zero and traffic stops. A resync runs only while the global quiescent signal is asserted (§11.9, hazard rules): with nothing in flight, a restored strap is never counted twice against data sent under the old one. 1. After the mux switches and the tile is quiescent, RAPIDS sends every cell and core a `set` credit packet, as one multicast, restoring its strap counter to what RAPIDS can now accept from it. 2. Each cell and core answers with a `set` advertisement of its current free input-buffer space. 3. RAPIDS rebuilds its counters from the answers. The saved legal-destination mask stands: a resync does not reopen the window, so it never waits on a destination that was illegal at boot.

A destination that leaves reset on its own uses the same path: its fresh advertisement replaces RAPIDS's view of it.

3.11.11 11.11 Cell-issued descriptors

A tetrad's control core (gatling) can make RAPIDS read memory and send the data to a cell by handing RAPIDS a complete source descriptor (owner direction 2026-10-18: gatling creates the descriptors). It is a pull: the core asks, the RAPIDS source half does the read and the send.

1. The core sends a `desc` packet to `level` 3 through RAPIDS's sink port: the header (tag = the source channel to run it on, length = 32) followed by one 256-bit RAPIDS source descriptor in the ordinary format. `src_addr` is the memory to read; `dst_addr` carries the destination coordinate and target address (RAPIDS HAS, Descriptor Format); `ptype`, `op`, `length` and the chaining fields mean what they always mean.
2. The sink port recognises `desc`, never stores it in a sink channel's SRAM, and passes the descriptor to the source half, which writes it straight into the tagged channel's descriptor queue, with no fetch.
3. An idle channel starts on it, as if kicked with the descriptor already fetched. A busy channel runs it after the descriptors already queued. A non-zero next pointer chains on into memory-resident descriptors as usual.
4. The source half reads `src_addr` and sends the data, with a header built from the descriptor, out of the source port to the destination it names.

Flow control (owner 2026-10-18): there is one descriptor queue, feeding the schedulers. `desc` packets land in an intake FIFO as deep as the descriptor credits handed out (a parameter); the intake FIFO is muxed into the put side of the descriptor queue, beside the memory fetch path. A core sends a `desc` packet only against its strapped share of those credits (§11.9, credit class 1), and RAPIDS returns an entry when the descriptor moves into the descriptor queue.

3.11.12 11.12 RAPIDS implementation status (2026-10-17)

Built in the byte-granular RAPIDS behind `TILE_HDR` = 1 (default 0; needs `SHARED_ENGINES` = 1 and `DATA_WIDTH` >= 256). RAPIDS Beats does not have it.

Item	State
Header packing (§11.5), payload types (§11.2)	Built: `rapids_pkg::tile_hdr_pack`. The header is the first beat of every packet on both ports, bits [127:0]; on an AXIS plane it is that first beat too, not `TUSER`
Source header from the descriptor (§11.7)	Built. Tag = the channel, source coordinate = the `TILE_SRC_COORD` parameter (level 3). A data packet's header leaves only when its first data beat is ready, so the two go back to back. A source descriptor with a `ptype` other than `cfg`/`prog`/`mem` is refused as an invalid descriptor
Read request (`op` = 1)	Built: a header-only packet, no memory read. As for a write, the descriptor completes when its packet is queued, not when the header has left; a zero-length request sends nothing
Sink header decode	Built (`rapids_tile_hdr_rx`): `cfg`/`prog`/`mem` writes are stripped and delivered by stream ID; `credit`, read requests into RAPIDS and reserved types are consumed, never stalled, and counted or flagged
Sink stream = tag	On RAPIDS's sink link the stream ID (`CXSSRCID`, the channel) must equal the header tag: the root sets it from the tag. RAPIDS delivers by stream ID and flags a mismatch (`sts_tile_tag_mismatch`)
Cell-issued descriptors (§11.11)	Built, with a one-descriptor slot per

Item	State
	source channel instead of descriptor grants: a `desc` with no free slot, one arriving during a channel reset, or an EXT one is dropped and sets that channel's sticky `sts_tile_desc_dropped` bit. The channel and the work it holds carry on
End-to-end credits (§11.9, §11.10)	Built behind `TILE_CREDITS` = 1 (2026-10-18): 64 destination counters in the source half, the internal and external ID of every node (`rapids_tile_credits`; index map §11.9), external counters filled by class-0 `set` / `add` credit packets from cells and cores; a packet to a cell or core leaves only when its counter covers every flit, header included. Node internal IDs are legal by topology, never held or charged, and their credit packets are ignored; a read request to a node is refused. The advertisement window (`SRC.TILE_ADV_WINDOW`, cycles from reset) closes into the saved 64-bit legal-destination mask (`SRC.TILE_LEGAL_DEST_LO` / `_HI`); tile descriptors wait for it and one to an illegal destination is refused as invalid. A channel reset does not refund credits already charged
Straps toward RAPIDS (§11.9, §11.11)	Not built: no per-cell landing space (`TILE_LAND_FLITS`), no `desc` intake queue, no credit returns from RAPIDS; RAPIDS ignores class-1 credits (rapids TASK-027)
Status	CSRs: `SNK.TILE_STATUS` and

Item	State
	SRC.TILE_DESC_DROPPED (W1C), SNK.TILE_COUNTS, and for credits SRC.TILE_LEGAL_DEST_LO / _HI / SRC.TILE_CREDIT_STATUS; the same values on the top-level sts_tile_* ports

3.11.13 11.13 Tetrad control core (owner direction 2026-10-17; deferred 2026-10-18; ruling 2026-10-19)

Deferral (owner direction 2026-10-18): the tetrad engine is defined **after the cell is fully defined and working** — the cell is the lowest module (cell PRD v0.3 §1). The owner-authored `gatling-rv32itetrad` barrel-core PRD (2026-10-17) is the leading candidate, with kestrel-class remaining a candidate flavor via `CORE_FLAVOR`.

Owner ruling 2026-10-19: gatling sequences the four **alu-infer** ALUs of its adjacent cells. Each barrel context's `POP` presents operands to the cell ALU's issue handshake, and `PUSH` retires result lines; the cell's slice-3 exec FSM remains the standalone bring-up driver with the same client contract. Gatling never originates config traffic.

Each tetrad carries **one RV32I control core** — kestrel-class, `CORE_FLAVOR`-swappable, defined with its full duty list in alu-infer PRD §5. The model, recorded here so the tile records it:

- **Attach.** The core sits at its tetrad node's **local port**, the same relationship the cell has with its own node. It is not a routing level, carries no coordinate of its own, and is reached as its tetrad: `prog` and `cfg` packets addressed `{level = 1, tetrad X/Y}` (§11.3) exit the tetrad node's local port into the core. Its firmware load rides the existing `prog` payload type (§11.2) — in-band like everything else.
- **In-band everything, no config origination.** All data, config, and programming for the tile blocks arrive with the delta traffic (owner ruling 2026-10-17). The core does not originate config traffic: the cells' programming (`FUNC_SEL`, `OP_LINES`, …) arrives in-band addressed to each cell (`level = 0`), carried by the stream. The core is a **local reaction agent**: it reads its four cells' status and IRQs over the local AXIL register plane, runs decision firmware (layer-boundary detection, flush policy), and exercises *operational* registers only (flush-and-send, soft reset, IRQ clear — the AXIL-for-registers plane, not fabric config).

- **Custom instructions stay minimal** (a custom-0 `KICK`/`WAIT` pair for local coordination, provisional pending the tetrad MAS); the `FLUSH` instruction is part of result-flush (§11.14). Encoding lands in the cell/tetrad MAS (§10).
- **Observability is unchanged.** SERV and the MonBus aggregation (§5) remain the tile's observability plane; the core reads registers and reacts — it takes no MonBus tap by default.
- **Why per-tetrad.** Four independent control points per tile (one per tetrad) allow layer pipelining across tetrads and one group-done per four cells, without sixteen instruction streams. A tetrad steps as 4 cells × 2 bf16 lanes = 8 bf16 lanes.

3.11.14 11.14 Result flush to the tile I/O (owner direction 2026-10-18; defined 2026-10-19)

Delta **must** be able to carry a cell's result data to `level` 3, the tile I/O, and so into RAPIDS. It is the path computed results leave the tile by:

1. **Detect.** Gatling, the tetrad engine (§11.13), detects a flush-to-memory condition. When to flush is program policy, not fixed hardware.
2. **Command.** Gatling sends a flush command to the result drain controller in each cell it applies to. **Gatling tells the cell the destination** (owner direction 2026-10-18): the cell does not choose it and does not need to know where RAPIDS attaches.
3. **Drain.** The controller pushes the cell's result memory out through the packetizer: write-format packets (`op` = 0) to `{ext 0, level 3}`, source = the cell's external ID `{1, 0, X, Y, Z}`, tag = the RAPIDS sink channel, each sent against the cell's `TILE_LAND_FLITS` strap (§11.9).
4. **Carry and land.** Delta routes each packet to the tile I/O. RAPIDS's sink strips the header and delivers the data by tag into that sink channel, which writes it to memory.

Flush instruction (owner ruling 2026-10-19): gatling PRD v0.2 adds the `Xcell FLUSH` custom instruction: `funct3 = 011`, `rs1[14:0]` = drain destination coordinate, `rs1[18:15]` = sink-channel tag. The instruction programs the cell's `FLUSH_DEST` / `FLUSH_TAG` and triggers the cell's flush-and-send. The coordination pair `KICK` (`funct3 = 100`) / `WAIT` (`funct3 = 101`) is also added (provisional pending the tetrad MAS). The engine is still deferred until the cell works (§11.13); the instruction encoding and cell CSR details land in the cell/tetrad MAS.

Link	State
Gatling detects and commands	Defined in gatling PRD v0.2: `Xcell FLUSH`, `KICK`, `WAIT`; engine deferred

Link	State
	until the cell works (§11.13)
Cell drain controller	Defined: the cell PRD has the packetizer and `CTRL` flush-and-send; no cell RTL beyond alu-infer
Delta to `level` 3	Interim: delta has no `level` field, so the destination gatling hands the cell includes the attach cell's delta coordinate, and the cell writes it into delta's header; delta needs no change for that. Permanent: `level` and `ext` in delta's header (delta DELTA-011)
RAPIDS sink delivery by tag	Built (§11.12)
Straps toward RAPIDS	Not built (rapids TASK-027). Without the reserved landing space a drain into a full sink channel stalls in the routers

3.11.15 11.15 Multi-path routing and failover: open hazards (2026-10-18)

Resilience is to come from delta route tables that hold a primary and an alternate next hop, with failover as an in-band table write and no new wires (delta DELTA-013). The tile topology already has the alternates: each tetrad node reaches the other three directly (K4) or through one of them, and every tetrad node links to both roots. Then a packet's path depends on table state, so a request and its reply can take different paths. The hazards, worked in DELTA-013, are recorded where they bite:

- **Delta** (delta HAS ch07, Resilient Routing: Open Hazards): mixed tables during a node-by-node update; the VC0 escape channel losing connectivity when a failed link sits on it, so the escape rule (up/down over this hierarchy is the natural one) is re-derived on every failover; the config route crossing the failure; and a misroute drop losing a `credit` or data packet, which breaks "always return credits" (§11.9).
- **Ordering is gone between packets.** Only the flits of one packet stay in order. Order becomes a requester-side property: a transaction ID per request, echoed in the reply. RAPIDS's cases (read replies sharing a sink-channel tag, a resync `set` racing an older `add`, a write followed by a packet that depends on it) are in the RAPIDS MAS, ch03 Tile Packet Header, Ordering Across the Fabric. Nothing in the tile may be built on fabric order.

- **Design fork:** fully adaptive on both legs, or pinned forward (requests follow the table's path, deterministic per table state) with an adaptive return. Pinned forward is the leading option, being much easier to verify and enough for surviving a forward-path failure; it still reorders at a failover flip and on the return leg.

3.12 12. Milestones

Owner direction 2026-10-19: the tile work is staged so that the cell and tetrad are proven before rapids and real data are brought in.

Milestone	What is proven
M1	A fully working **tetrad** on the FPGA — 4 cells + gatling sequencer over delta; proves the cell datapath, credit loop, and programmability.
M2	A **tile** running in isolation (with injectors) on an FPGA — 16 cells, tetrads, roots, and MonBus levels; no rapids.
M3	**Rapids** + **tile** working artificially (injectors/test data) — full source/sink loop with tile headers, end-to-end credits, and straps.
M4	**Rapids** + **tile** working on real data and benchmarks.

3.13 References

- `docs/superpowers/specs/2026-10-13-delta-mesh-design.md` — working design record (§2 architecture, §5.4 mode catalog).
- `3d-mesh-tile.drawio` — owner's K4-at-both-scales figure.
- `delta_has/ch07_mesh_network/` — the node/network chapters in delta's HAS (cell/flit, routing, flow control, QoS, multicast, modes, observability, verification).
- `noc-ip/delta_spec` — the aspirational NoC draft this subsystem grew out of.

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

← Back to compute-eng-ip

4 ALU-INFER — Cell Inference ALU Product Requirements Document

Version: 0.50 (2026-10-18 — **revs pinned at 0.50 per owner, "for now,"** while the ALU heads to the board; this rev accumulates every ruling to date). Accumulated since the v0.3 base: **tag-based elastic alignment** (§4 invariant 3) — each loaded invocation takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks tags in flight; an op asserts its valid only when every op of its tag group is ready; tags stay inside the block and the seam keeps its two valid/ready pairs only; enables the §7 growth item (deep-pipelining the multiplier / FMA rows with the adder's `PIPE_STAGE_*` pattern). **Tetrad-engine deferral / gatling ruling 2026-10-19** (§5) — the per-tetrad core is defined after the cell is fully defined and working; the owner-authored gatling-rv32itetrad barrel-core PRD is the **selected sequencer** for its tetrad (kestrel-class stays a `CORE_FLAVOR` candidate). **Port-width correction** — operand ports stay `LINE_W` each (one line per side, both captured in the same clock); the earlier "widening" was a line-vs-lane mis-count. **Owner accept-and-defer** — M7 and `acc_ctrl` accepted as drafted; per-row latency work deferred to FPGA bring-up (M8). **Elastic-pipeline interface** — valid/ready at both ends, backpressure freezes the alignment tracking, II=1 while draining, no padded stages on quick rows, in-order results. **Base decisions** — bf16 element model on the rtl/math `math_bf16_*` library, fp32 MAC accumulate, one RV32I core per tetrad with in-band everything. **Status:** Pre-HAS PRD, and the **normative definition of the cell's ALU block**: the RTL implementation lives in this component (`alu-infer/rtl/`, unit tests in `alu-infer/dv/tests/`; moved from `cell/rtl/` 2026-10-20) and the **cell** component integrates it. Where this document and the cell PRD touch the ALU, this one governs (the cell PRD was reconciled to it 2026-10-17; see §2).

4.1 1. Identity and scope

alu-infer is the compute engine of the **cell**: the function array the cell's execution FSM drives to turn input-memory lines into result-memory lines. It is an *inference* ALU — the arithmetic a neural-network layer needs (add, multiply, fused accumulate, min/max, threshold, ReLU-family activations), not a general-purpose ISA datapath.

Component base (owner direction 2026-10-17): the implementation is built mostly from the `math_bf16_*` family of `rtl/math/` — the repo's existing brain-float arithmetic library. Nearly every v1 function row has a block there (adder, multiplier, FMA, comparator, min/max, ReLU, and the rest of the activation set), so the ALU is the math library's inference consumer the same way the falcon suite's garuda rung is its IEEE-fp32 consumer. Rows without a bf16 block (copy, and the compare *mask* packing) are wires and glue.

Placement (owner decision 2026-10-17, updated 2026-10-20): this directory holds the product definition, verification doctrine, the RTL (`rtl/`, SystemVerilog), and its cocotb unit tests (`dv/tests/`) — organized like the repo's other component areas (the dma-ip/stream pattern: flat RTL with per-block filelists and a lint master, a four-line area Makefile, and a discoverable `dv/tests` suite). The `cell/` component integrates the block as the cell's "ALU + execution FSM"

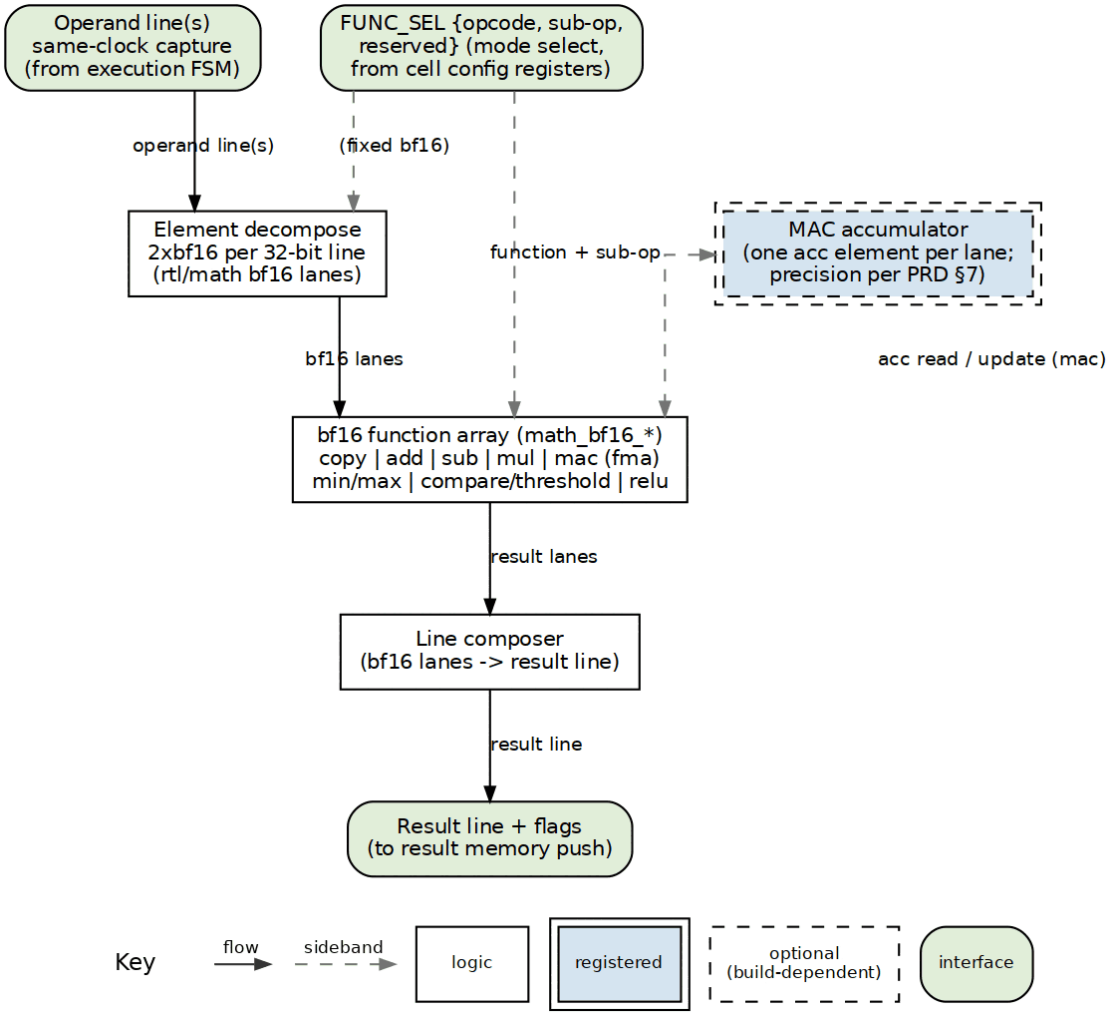
element: the cell filelists include this block's filelist, and cell-level tests will verify it with the memories, packetizer, and credit logic it serves. The block boundaries in the cell PRD were chosen so a future CGRA array replaces this ALU without disturbing the network, credit, or observation contracts (cell PRD §11); this PRD's interface contract (§4) is written to that same seam.

Out of scope: the execution FSM's sequencing policy, the memories, the packetizer, credit/grant logic, and the delta node (all cell PRD territory).

4.2 2. Element model

Owner direction 2026-10-17: a **32-bit line is read as 2×bf16 elements** — two 16-bit brain-float lanes — and the function array is built from `math_bf16_*` blocks. bf16 keeps fp32's 8-bit exponent and truncates the mantissa to 7 bits, which is why the library's inference blocks (and the quantization literature behind them) chose it: full fp32 range at half the storage. The packed-line structure of cell PRD §4 is retained: the configured function applies to every element in the line **in parallel** (2 lanes per 32-bit line), and wider lines (cell PRD §12) scale the lane count, never the element width.

Supersession, reconciled. Cell PRD §4's element-width note (owner, 2026-10-16) defined B4/B8/B16/B32 *integer* widths; that note is superseded by this bf16 model. The cell PRD was updated to match on 2026-10-17 — the B4–B32 integer model is not planned (a companion integer mode is deferred, §7). One source per fact: this PRD owns the ALU's element model; the cell PRD links here rather than restating it.



ALU-INFER datapath

4.3 3. Function set (v1)

Opcode rows on the bf16 base; the `math_bf16_*` block named in each row is the implementation base in the ALU's RTL. Each row is an RTL table entry, not a new block.

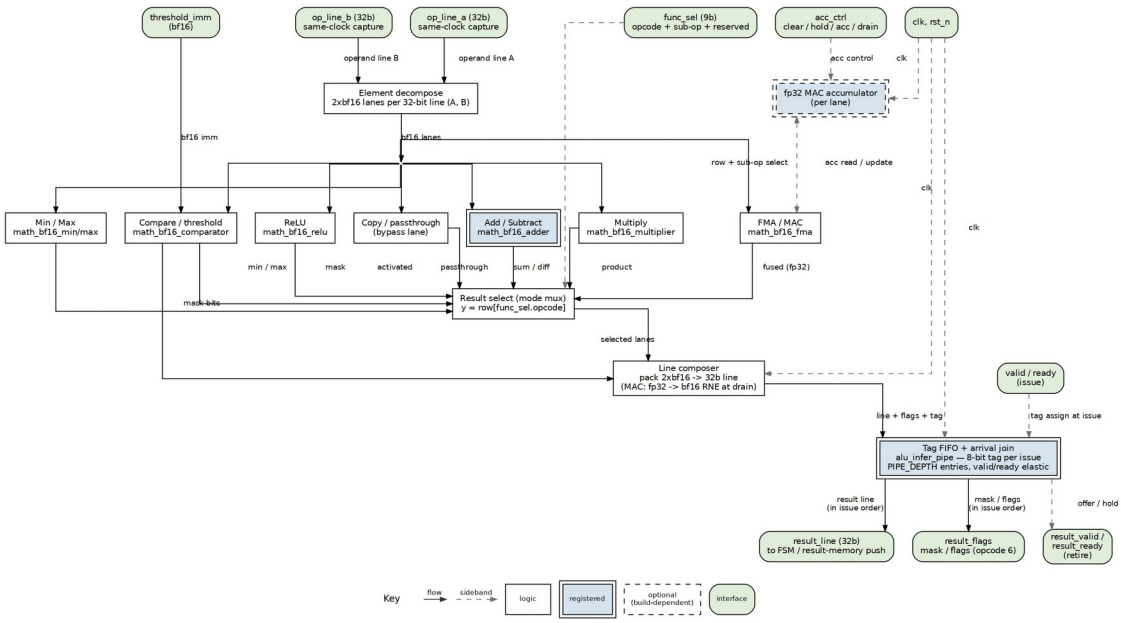
Opcode	Function	`rtl/math` base	Operand pattern
0	copy / passthrough	wires	1 line → 1 line
1	add	`math_bf16_adder`	2 lines → 1 line
2	sub	`math_bf16_adder` (sign-flipped operand)	2 lines → 1 line

Opcode	Function	`rtl/math` base	Operand pattern
3	mul	`math_bf16_multiplier` (`math_bf16_mantissa_mult` + `math_bf16_exponent_adder`)	2 lines → 1 line
4	mac	`math_bf16_fma` — fused, fp32 accumulate (§3 note)	2 lines + accumulator → 1 line
5	min / max (sub-op 1b)	`math_bf16_min` / `math_bf16_max` (via `math_bf16_comparator`)	2 lines → 1 line
6	compare / threshold (sub-op)	`math_bf16_comparator`	2 lines or line + imm → 1 mask line
7	relu	`math_bf16_relu`	1 line → 1 line

MAC accumulation (decided 2026-10-17): the FMA row computes the bf16 product **exactly in fp32** (8×8 significand bits fit fp32's 24) and accumulates in a **per-lane fp32 accumulator**; one rounding to bf16 per result line at drain. This is the production bf16 pattern; it promotes the v0.2 open question to a decision.

Opcode space reserves 5 bits; sub-ops 2 bits; 8–31 are growth. The fused family (`math_bf16_fma`) makes the mac row the workhorse an inference workload expects.

The full block diagram — every v1 row with its `rtl/math` block, the fp32 accumulator, and the sideband plane:



ALU-INFER block diagram

Growth rows (same table, later opcodes — the library blocks already exist): leaky-relu / gelu / sigmoid / silu / tanh activations (`math_bf16_leaky_relu`, `math_bf16_gelu`, `math_bf16_sigmoid`, `math_bf16_silu`, `math_bf16_tanh`); reciprocal and divide (`math_bf16_reciprocal`, `math_bf16_newton_raphson_recip`, `math_bf16_goldschmidt_div`); exp2 / log2 scaling (`math_bf16_exp2`, `math_bf16_log2`, `math_bf16_log2_scale`); int8 drain (`math_bf16_scale_to_int8`); format conversions (`math_bf16_to_fp32`, `math_bf16_to_fp16`, `math_bf16_to_fp8_e4m3/e5m2`).

`FUNC_SEL` (cell PRD §7) carries {opcode 5b, sub-op 2b, element-width 2b} — the element-width field is **reserved** under this model (bf16 is the only v1 element width); its final disposition is §8.

4.3.1 Function-set roadmap (owner ruling 2026-10-19)

The v1 rows 0–7 in the table above stand unchanged. Row additions after FPGA bring-up, in priority order:

1. **reduce** — sum/max tree over a span of lines into the fp32 accumulator; unlocks softmax/layernorm/pooling.
2. **PWL/LUT activation** — 256-entry LUT + linear interpolation covering sigmoid/tanh/gelu/exp; exp plus reciprocal-multiply covers softmax's divide. No true divider is planned.
3. **const/bias** — emit the immediate register as a result line so programs can build bias lines without memory traffic.

Format roadmap, via the reserved `func_sel[8:7]` element-width field (open item M6): **int8** (4 lanes per 32-bit line) first — the same MAC fabric runs 4 MACs/clk and teaches quantization; **fp32** (1 lane per line) second — a general vector engine. **bf16 stays the v1 format.**

No-fork rationale: there are no A/B/C ALU variants — one ALU per cell; heterogeneity belongs at cell/tile granularity because delta/tile is payload-agnostic. The `alu-rv32i` and `alu-rv32f` blocks are ISA-datapath ALUs for the falcon cores (a different role, not variants of this one).

4.4 4. Interface contract

The contract the cell implementation obeys at the ALU↔FSM seam:

Signal group	Direction	Width	Notes
`op_line_a`, `op_line_b`	in	LINE_W each	operands — the function's `OP_LINES` count is across both ports (1 for copy/relu: the A line only; 2 for binary/mac: the A line and the B line); both lines are presented **in the same clock** and captured at the issue edge (owner ruling 2026-10-17: two ops scheduled in the same clock finish at the same time)
`threshold_imm`	in	16 (bf16)	immediate operand for opcode 6
`func_sel`	in	9b	{opcode, sub-op, reserved} — the legacy element-width field is

Signal group	Direction	Width	Notes
			reserved (§3)
`acc_ctrl`	in	—	accumulator clear / hold / accumulate (opcode 4 state)
`result_line`	out	LINE_W	one line per invocation for all v1 rows (2 bf16 lanes, fp32-rounded at drain for opcode 4)
`result_flags`	out	LINE_W	opcode-6 mask — one bit per lane, packed low (M5 closed 2026-10-17, owner ruling); zero for other rows
`valid` / `ready`	in handshake	1/1	elastic issue: the invocation enters the row pipeline when `valid && ready`; `ready` deasserts only under downstream backpressure, so a new invocation may be accepted **every clock** while the output drains (II=1); the FSM holds `valid` and its operands/config stable while `ready`

Signal group	Direction	Width	Notes
			is low
`result_valid` / `result_ready`	out handshake	1/1	elastic retire: each result is offered exactly `ROW_LATENCY` cycles after its issue (continuous drain) and held with stable data until accepted; results retire strictly in issue order

Invariants:

1. **Lane independence.** Except for opcode 4's accumulator, no lane of a result line depends on any other lane of the same or the other operand line. A lane can be checked against a scalar golden reference in isolation.
2. **Accumulator isolation.** The fp32 accumulator is per-lane, private to opcode 4, cleared only by `acc_ctrl`.
3. **Registered shell, uniform presentation, elastic timing.** Rows may be combinational or pipelined (cell MAS decides); the contract is a valid/ready elastic pipeline either way: every result is **presented** exactly `ROW_LATENCY` cycles after its issue, in issue order, when the output drains continuously. Quick rows compute in parallel at their natural speed and the block's alignment mechanism delays their presentation — no padded pipe stages on fast rows. **Tag-based alignment (owner ruling 2026-10-18):** when an invocation is loaded it takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks the tags in flight, and an op asserts its valid only when every op of its tag group is ready — the join yields group-aligned, in-order, elastic retirement without per-op timers. The v1-as-built block realizes the same contract with the alignment-timer mailbox of the MAS; the tag mechanism is the alignment architecture when rows gain uneven pipeline depths (§7). Backpressure propagates: a stalled consumer holds `result_valid` (data stable), freezes the alignment tracking, and deasserts `ready` at the input once the pipe fills; on release the pipeline

resumes where it stopped. **Scheduler-paced feedback.** Whenever results feed back — opcode 4's chain today, any future recurrence — the scheduler owns the pace entirely: it shapes the `valid` stream and obeys `ready`, and the ALU makes no autonomous pacing decisions (owner ruling 2026-10-17). **No tags on this seam** — the tags above live inside the block; seam flow control is the two valid/ready pairs alone, with no error signal. Steady state, un-stalled, is one invocation per clock regardless of depth.

4. **Function-table growth.** New opcodes and sub-ops are additive rows; they must not change §4's handshake or the lane-independence invariant.

4.5 5. Control interface — one RV32I core per tetrad, in-band everything

Owner ruling 2026-10-19: gatling sequences the four ALUs of its adjacent cells. The barrel RV32I core is the programmable sequencer for its tetrad — each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. Gatling is also a barrel RISC-V with custom commands (Xcell: CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT) — the program is the schedule. Gatling never originates config traffic; cell configuration arrives in-band over delta.

The tile blocks take **all data, config, and programming in-band on delta** (owner ruling 2026-10-17). The programming stream rides with the data as `cfg` packets (cell PRD §3); the cells' demux writes the §7 register map from the stream — **the stream is the programmer**. Consequences, decided with the owner:

- **The tetrad controller is deliberately not a config originator.** No config traffic is generated by the compute-side control core; configs arrive inline with the delta traffic. There is no per-tetrad config broadcaster to design or verify.
- **One RV32I core per tetrad** (4 cells), four per tile: the local programmable agent for its tetrad. The owner-authored `gatling-rv32itetrad` barrel RISC-V is the tetrad sequencer (owner ruling 2026-10-19). **Kestrel-class** — single-cycle, CPI 1, the repo's proven core (rv32ui battery and riscv-formal green) — remains a `CORE_FLAVOR` candidate via the same swap pattern HIVE-C uses. **Not SERV:** SERV is the repo's bit-serial *agent* core, and its role is the Ocelus observability infrastructure (hive-serv monitors) — its CPI class is wrong for compute control, and its job is already taken.
- **Duties** — local reaction, not programming: read cell status and IRQs (done, error, grant-starved, credit-timeout — cell PRD §8) over the local AXIL

register plane; run small decision firmware (layer-boundary detection, flush policy — the cell PRD §12 open item; local event bookkeeping); exercise *operational* registers (flush-and-send, soft reset, IRQ clear — the AXIL-for-registers plane, not fabric config). Its own firmware arrives in-band like everything else's, booted through the repo's loader-window pattern.

- **Custom instructions stay minimal** (gatling Xcell subset in the custom-0 opcode space): KICK (funct3=100) starts its tetrad's cells once local conditions hold, WAIT (funct3=101) stalls until the tetrad's group-done bitmask is set; POP/PUSH present operands and retire results across the ALU↔️FSM seam. The gatling PRD v0.2 carries the exact encodings; the ALU's elastic handshake and tag-alignment contract are unchanged. The array's programming model remains the in-band stream; the core's firmware programs only its own reactions.
- **Why per-tetrad:** the tetrad is the tile's structural group; one group-done per four cells; four independent control points allow layer pipelining across tetrads without sixteen instruction streams or sixteen I-mems. SIMD width per tetrad: 4 cells × 2 bf16 lanes = 8 bf16 lanes stepping together; the 16-cell tile is 32 bf16 lanes.
- **Observability stays with Ocelus.** The tile's MonBus aggregation and the SERV-based monitor plane are untouched; the tetrad core reads registers and reacts — it is control, not instrumentation (§8 lists the boundary as an open detail).
- **Deferral (owner direction 2026-10-18):** the tetrad engine — core flavor, attach, and duties above — is defined **after the cell is fully defined and working**. The cell is the lowest module (cell PRD v0.3 §1); the owner-authored `gatling-rv32itetrad` barrel-core PRD (2026-10-17) is the selected sequencer, with kestrel-class remaining a candidate flavor via `CORE_FLAVOR`. The in-band-everything model in this section is unchanged by the deferral.

4.6 6. Verification hooks

- Stack per house convention: cocotb + rds-dv BFMs (cell PRD §9); the ALU's unit tests live in the cell's `dv/` tree since the RTL does.
- **Golden reference:** a small bf16 numerics model (fp32 computation per lane, rounded to bf16 RNE) — matching how the math library's own dv checks the bf16 blocks; lanes compared independently, which §4 invariant

1 makes a strong check. Opcode 4 checks the fp32-accumulate chain (long MAC runs, clear/hold corners, single-round-at-drain).

- **Special values are mandatory corners**, not extras: NaN payloads, ±inf, ±0, and denormals/FTZ behavior (the library's `SUBNORMAL_SUPPORT` parameter lineage — see the math references index) on every arithmetic row.
- **Activation rows** check against the library's reference approximations (piecewise/LUT), with the accuracy class stated per row in the cell MAS.
- **Cross-check against `rtl/math`:** the ALU's rows instantiate the library blocks; a unit-vs-library diff on identical operands is a bug in exactly one of them.
- **Controller bring-up** reuses the kestrel battery, loader contract, and lockstep flow unchanged (the HIVE-C precedent); the kick/wait extension and the in-band cfg stream are verified at the cell testbench level — the cfg path already exists in the cell PRD's dataflow tests.
- Coverage matrix: opcode (8) × special-value class × lane position (2 per line), plus accumulator chains and threshold immediate corners. The cell PRD's credit-loop tests already verify lines-in → credits-out independently of the datapath.

4.7 7. Deferred and growth

- **Deep-pipelined multiplier / FMA rows** — the combinational fp32 FMA (with its accumulator feedback) is the timing risk at FPGA bring-up; pipeline it with the adder's `PIPE_STAGE_*` pattern. Mixed row depths then stay II=1 with in-order retirement under the tag-based alignment ruling (§4 invariant 3), which "naturally takes care of the elastic nature" (owner 2026-10-18). A single lane's serial MAC dependency is arithmetic, not implementation — pipelining does not change it.
- **Activation expansion** (gelu, sigmoid, silu, tanh, leaky-relu) — library blocks exist; table rows once an owner priority calls for them.
- **Reciprocal / divide, exp2 / log2, int8 drain, format conversions** — library blocks exist (§3 growth list); same path.
- **fp8 element lanes** — `math_fp8_e4m3_*` / `math_fp8_e5m2_*` families exist; a mixed bf16/fp8 mode is an element-model extension, not v1.
- **Companion integer mode** (B4–B32) — deferred, and not currently planned; the bf16 model is the accelerator's single datapath (§2).

- **CGRA replacement** — per cell PRD §11, an array replaces this ALU behind the same FSM handshake.
- Context switching / multi-function pipelines and cell-initiated grant requests (both cell PRD §11) — cross-referenced, not restated.

4.8 8. Open questions

- **NaN / inf semantics in min/max and activations.** IEEE 754-2008 `minNum/maxNum` quiet-NaN propagation vs the inference-convention "ignore NaN" — the library's `math_bf16_min/max` and `math_bf16_comparator` semantics decide; state the choice in the cell MAS.
- **Rounding and exception policy.** RNE throughout per the library; a dynamic rounding mode is out of scope (garuda's RNE ruling is the precedent). Whether the ALU reports bf16 flag equivalents (the library's blocks carry IEEE exception outputs) into `result_flags`, or leaves them unobserved, is a cell-observability decision.
- **Denormal / FTZ behavior** on the arithmetic rows — follow each library block's `SUBNORMAL_SUPPORT` default, or force one policy cell-wide?
- **Tetrad-controller boundary details** — the core's program-load path and local memory map, the kick/wait encoding, and whether it ever takes a MonBus observation tap (default: no — Ocelus/SERV owns observability). All land in the cell/tile MAS, not this PRD.
- **Immediate encoding** for opcode 6 (`threshold_imm` as a raw bf16 bit pattern vs a small int compared after convert).
- **Result-flag line format** for opcode 6 — **closed 2026-10-17** (owner ruling): per-lane 1-bit mask, packed low in the `LINE_W` line (M5).
- **`FUNC_SEL` element-width field disposition** — retire the field from the register map, or keep it reserved for the fp8/integer growth modes.

4.9 9. Dependencies

- `rtl/math` `math_bf16_*` family — the implementation base (§3 names the block per row): adder, multiplier (`mantissa_mult`, `exponent_adder`), `fma`, `comparator`, `min`, `max`, `relu`, plus the activation, divide, scale, and conversion blocks of the growth list.
- `cell/docs/PRD.md` — the cell around the ALU (memories, FSM seam, packetizer, credits; reconciled to this PRD 2026-10-17).

- `kestrel-rv32i/` — the tetrad controller's reference core (falcon suite; `CORE_FLAVOR`-swappable per the HIVE-C precedent).
- `tile/docs/TILE_SPEC.md` §11 — packet, credit/grant, and coordinate protocols the cell (not the ALU) answers to; listed so a reader of this PRD can find the whole contract. (The tetrad-level control core is not yet recorded in the tile spec — flagged for its next edit.)
- `rtl/math/references/README.md` — the central math-paper index (bf16 format, activation, quantization, and fp8 citations for §§3, 7).

4.10 10. References

- `cell/docs/PRD.md` — the cell around the ALU (normative for everything except the ALU block, which this PRD owns).
- `riscv-ip/README.md` — the falcon suite (kestrel rung: the tetrad controller's reference core; `CORE_FLAVOR` precedent per the hive-serv PRD §10.3).
- S. Wang, P. Kanwar, "BFloat16: The Secret to High Performance on Cloud TPUs," Google Cloud blog, Aug. 2019 — the bf16 format's canonical definition (non-peer; citation-only in the math index). `rtl/math/references/README.md`.
- V. Nair, G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," *Proc. 27th ICML*, 2010 — backs the opcode-7 ReLU row. PDF: `rtl/math/references/ReLU__Nair2010.pdf`.
- D. Hendrycks, K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv, 2016 — backs the §7 gelu growth row. PDF: `rtl/math/references/GELU__Hendrycks2016.pdf`.
- S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units…" (*Neural Networks*, 2018) and P. Ramachandran, B. Zoph, Q. V. Le, "Searching for Activation Functions" (Swish, arXiv 2017) — back the §7 silu growth row. PDFs: `rtl/math/references/SiLU__Elfwing2018.pdf`, `rtl/math/references/Swish__Ramachandran2017.pdf`.
- B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proc. IEEE CVPR*, 2018 — backs the §7 int8-drain row (`math_bf16_scale_to_int8`) and the §3 int8-lane roadmap. PDF: `rtl/math/references/Int8Quantization__Jacob2018.pdf`.
- D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning," arXiv, 2019 — backs the bf16 format choice (secondhand citation; primary provenance

is the Google Cloud blog and the IEEE 754-2008 fp32 exponent). PDF: `rtl/math/references/README.md`.

- The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA — the custom-0 opcode space is the defined home for the gatling Xcell instruction subset (KICK/WAIT/…) described in §5. https://riscv.org/technical/specifications/

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 ALU-INFER Micro-Architecture Specification Index

Component: ALU-INFER — the cell's bf16 inference ALU (the cell PRD's "ALU + execution FSM" block, implemented inside the cell component) **Version:** 0.50 (pinned — revs held at 0.50 per owner, "for now," while the ALU heads to the board) **Date:** 2026-10-17 **Purpose:** Detailed micro-architecture specification for the ALU-INFER block: what the datapath actually does, block by block

---

5.1 Overview

ALU-INFER is the compute engine of the **cell**: a bf16 function array that the cell's execution FSM drives with a pop → compute → push loop, turning input-memory lines into result-memory lines under a config-selected function. It is built mostly from the `math_bf16_*` family of `rtl/math` — the adder, multiplier, FMA, comparator, min/max, and ReLU blocks are instantiated, not reinvented — and it holds exactly one piece of state: a per-lane fp32 MAC accumulator. This is the internals document, the one you open when you need to know what the datapath does rather than what the product definition (PRD) requires of it.

5.1.1 Key Features

- **bf16 packed lanes:** a 32-bit line is two bf16 elements; every row computes both lanes in parallel, independently
- **Library-instantiated rows:** each of the seven v1 rows names its `rtl/math` block (`math_bf16_adder`, `math_bf16_multiplier`, `math_bf16_fma`, `math_bf16_comparator`, `math_bf16_min`, `math_bf16_max`, `math_bf16_relu`) — structure, latency, and special-value behavior are inherited from the library's own documentation and dv
- **fp32 MAC accumulation:** bf16×bf16 products are exact in fp32, so the FMA row accumulates in fp32 and rounds to bf16 once per result presentation — the production pattern, decided 2026-10-17
- **Function-table growth:** opcodes 8–31 and new sub-ops are RTL table rows behind the same valid/ready elastic pipeline; activations, reciprocal/divide, and conversions are the queued growth set
- **A hard boundary:** the ALU originates no config traffic (the stream is the programmer), exposes no monitor taps (observability is Ocelus's job), and speaks only to the cell FSM across one interface

5.1.2 Design Philosophy

The math library is the datapath. New arithmetic lands in `rtl/math` first and is consumed here second; the ALU is the library's inference consumer the way the falcon suite's garuda rung is its IEEE-fp32 consumer.

FMA-first. Inference time is multiply-accumulate time; every other row is sized to sit beside the FMA without dwarfing it, and the one piece of state in the block exists for the FMA's accumulator.

A function table, not a function zoo. The result select is a port-driven mode mux — no arbitration, no starvation — and the house block diagram correctly shows a plain box, not a trapezium.

5.2 Design Notes

5.2.1 Version History

Rev 0.50 (2026-10-18, current — revs pinned at 0.50 per owner, "for now," while the ALU heads to the board): tag-based elastic alignment ruling (PRD v0.50) — each loaded invocation takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks tags in flight; an op's valid asserts only when every op of its tag group is ready. **Implemented 2026-10-19** (func+gate green, 13 cocotb tests × {gate,func} × `PIPE_DEPTH` {4,8}): `alu_infer_pipe` is the tag FIFO with an arrival join; results retire in issue order at natural per-row latency; a stalled consumer no longer blocks issue until the FIFO fills; `ROW_LATENCY` is replaced by `PIPE_DEPTH` (default 8, [2,16]) and `ADDER_LATENCY` (default 3, [0,4]). The PRD's "exactly `ROW_LATENCY` cycles" phrasing described the retired mailbox realization; the seam contract is unchanged — tags stay inside the block. Carries Rev 0.5's port-width correction.

Rev 0.5 (2026-10-17): port-width correction (PRD v0.9) — the v0.7 "widening" of `op_line_a/b` to `OP_LINES × LINE_W` was premised on a mis-count of operand lines vs lanes; a binary/mac invocation pops one line per side (OP_LINES counts the A+B pair), so the ports stay `LINE_W` each. The owner ruling underneath — both operand lines in the same clock — is unchanged. Diagram port labels re-rendered. Carries Rev 0.4's accept-and-defer pass and Rev 0.3's rulings.

Rev 0.4 (2026-10-17): owner accept-and-defer pass — M7 and the `acc_ctrl` encoding accepted as drafted ("works for now"); per-row latency work deferred to FPGA bring-up (M8), where aggressive per-function latency minimization is expected — the alignment-timer contract absorbs faster rows without interface churn (noted at `alu_infer_pipe`). PRD §8's result-flag question closed (packed low). Carries Rev 0.3's rulings.

Rev 0.3 (2026-10-17): owner rulings closing the implementation gaps (PRD v0.7) — operand ports are `OP_LINES × LINE_W` so all operand lines present **in the same clock** and are captured at the issue edge (Waveform 3.1 redrawn; block-diagram port labels updated); **scheduler-paced feedback**: the ALU makes no autonomous pacing decisions; CLEAR flows through the pipe and retires a don't-care slot; an errored opcode retires its slot behind the IRQ; `result_flags` closed — one mask bit per lane, packed low (**M5 closed**).

Rev 0.2 (2026-10-17): elastic-pipeline interface (owner direction, PRD v0.6) — the ALU↔FSM seam is documented as a valid/ready elastic pipeline: `valid`/`ready` issue handshake, `result_valid`/`result_ready` retire handshake, backpressure propagates upstream and freezes the alignment timer; Waveform 3.1 redrawn to show backpressure at both ends. Carries Rev 0.1's decisions unchanged.

Rev 0.1 (2026-10-17): initial MAS, drafted alongside the PRD v0.3 decisions — bf16 element model on the `math_bf16_*` library, fp32 MAC accumulation (presentation semantics decided as M7), per-tetrad control interface, in-band config/programming. v1 RTL landed in `cell/rtl/` on 2026-10-17/18 (func+gate green) and moved to `alu-infer/rtl/` on 2026-10-20; chapters 2–4 document the as-built implementation and carry the open decisions M1–M8 of Chapter 6 in their labels.

5.3 References

5.3.1 Visual Assets

Diagram and waveform sources and their renders live in `assets/`:

- **Source Files:**
  - `assets/graphviz/*.dot` — Graphviz block-diagram sources (derived from the PRD-canonical copies in `../../assets/graphviz/`)
  - `assets/wavedrom/*.json` — WaveDrom timing-diagram sources
- **Rendered Files:**
  - `assets/graphviz/*.png` — Rendered block diagrams
  - `assets/wavedrom/*.png` — Rendered timing diagrams

`gen_alu_infer_figures.py` (one directory up) regenerates the waveforms and copies the diagrams; each `graphviz/` directory's `regenerate_all_graphviz.sh` re-renders the `.dot` files.

5.3.1.1 Diagrams

1. **Datapath overview** — assets/graphviz/01_alu_infer_datapath.png
2. **Full block diagram** — assets/graphviz/02_alu_infer_block.png

5.3.1.2 Waveforms

1. **Elastic handshake (backpressure at both ends)** — assets/wavedrom/wvf_3_1_invocation_handshake.png
2. **Back-to-back issue (streaming)** — assets/wavedrom/wvf_3_2_back_to_back.png
3. **MAC chain** — assets/wavedrom/wvf_4_1_mac_chain.png

5.3.2 Companion Specifications

- **alu-infer PRD** — product definition (normative for the ALU block; this MAS implements it)
- **cell PRD** — the cell around the ALU: memories, FSM seam, packetizer, credit logic, §7 register map

5.3.3 Project-Level

- **README:** ../../README.md — component pointer page
- **TILE_SPEC §11.13:** ../../../tile/docs/TILE_SPEC.md — the tetrad control core's place in the tile
- **KESTREL MAS:** ../../../../riscv-ip/kestrel-rv32i/docs/kestrel_mas/kestrel_mas_index.md — the tetrad control core's reference core
- **rtl-math book:** ../../../../../../docs/markdown/rtl-math/index.md — the library the rows instantiate

5.4 Navigation

5.4.1 Document Organization

Each chapter below is one source file.

5.4.1.1 Front Matter

- Document Information

5.4.1.2 Chapter 1: Overview

- Design Philosophy and Datapath Organization

5.4.1.3 Chapter 2: Functional Blocks

- alu_infer_top (module tree)
- The Seven Function Rows
- The FP32 MAC Accumulator
- Result Select and Line Composer

5.4.1.4 Chapter 3: Internal Interfaces

- The ALU↔FSM Interface

5.4.1.5 Chapter 4: Micro-Architecture Behavior

- The Compute Cycle, per Opcode Class

5.4.1.6 Chapter 5: Verification Hooks

- Verification Hooks and Observability

5.4.1.7 Chapter 6: Configuration

- Parameters, Encodings, and Open Items

5.4.2 Quick Navigation

5.4.2.1 For New Users

1. Start with Overview for the design commitments and the dataflow in one paragraph
2. Read the full block diagram chapter for the module tree and parameters
3. Study the ALU↔️FSM interface for the handshake that brackets every invocation
4. Reference Configuration for encodings and the open-items register

5.4.2.2 For Implementers

- **A row's datapath:** see Function Rows for the library block per opcode and its wiring
- **The accumulator:** see FP32 MAC Accumulator for CLEAR/ACCUMULATE/DRAIN semantics and isolation invariants
- **Timing:** see Compute Cycle with the MAC-chain waveform
- **Testbenches:** see Verification Hooks for the golden model, the built-in library cross-check, and the coverage matrix

---

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Document Information

6.1 Overview

6.1.1 CELL Micro-Architecture Specification

Document information

Property	Value
Document Title	CELL Micro-Architecture Specification
Version	0.13
Date	2026-10-20
Status	Draft — slice-1 RTL built; slice-2/3 modules specified
Classification	Open Source - MIT License

6.1.2 Document Purpose

This Micro-Architecture Specification (MAS) documents how the compute cell is built and how the remaining slice-2/3 work completes it:

- The slice-1 module tree (`cell_top`, `cell_rx`, `cell_cfg_regs`, `cell_mem_in`, `cell_mem_out`) as it exists today
- The slice-2/3 modules (`cell_credit`, `cell_packetizer`, `cell_exec_fsm`) that are specified here
- The MAS decisions that close the PRD §12 open questions
- The cycle-level behavior of the fill path as built and the expected consume/emit path
- The verification plan for the credit loop and ALU-run tests

For what the cell must do and why, see the **cell PRD** (normative for the cell product). For the packet, credit, and coordinate protocols the cell consumes, see **TILE_SPEC §11**. For the ALU block itself, see the **alu-infer PRD** and **alu-infer MAS**.

6.1.3 Intended Audience

- RTL designers implementing the slice-2/3 modules inside `cell/rtl/`
- Verification engineers extending `cell/dv/tests/test_cell.py`
- Design reviewers checking the future implementation against intent
- Students reading the cell datapath top to bottom

6.2 Design Notes

6.2.1 Revision History

Revision history

Version	Date	Author	Description
0.11	2026-10-18	RTL Design Sherpa	Beat-chained header rule (owner ruling): the 128-bit tile header occupies the first `ceil(128/DW)` payload beats, LSB-first; every link keeps its own width. Local port stays at delta `DATA_WIDTH` (64b) — supersedes 0.10's 128-bit local-port widening. TILE_SPEC v1.6+ binding.
0.12	2026-10-19	RTL Design Sherpa	(addendum) Owner rulings 2026-10-18/19: local-port width stays at delta DATA_WIDTH (64b) with beat-chained header; strap terminology replaces grants in registers/PRD; DEST reserved, FLUSH_DEST/FLU

Version	Date	Author	Description
			SH_TAG are the operational drain destination; credit-class encoding added; gatling sequences the cell ALU. Slice-1 RTL identifiers retain pre-ruling grant names (noted in ch06).
0.11	2026-10-18	RTL Design Sherpa	(same-day addendum) ALU-focus datapath figure (fig 2.3, no delta side) + issue-tag / same-tag-alignment MAS decision in ch04 §4.4 — ratified owner ruling; tag assign/FIFO/align gate not yet in alu-infer RTL.
0.10	2026-10-18	RTL Design Sherpa	First issue. Slice-1 RTL built; slice-2/3 specified. Closes PRD §12 open questions with MAS decisions: IN_DEPTH=OUT_DEPTH=32 lines; CREDIT_BATCH_N default 1; 64-cycle partial-batch credit timeout;

Version	Date	Author	Description
			EMIT_BATCH_X flush on batch-full or CTRL.flush-and-send; strap counter default 16; local-port data width widens to 128 bits to carry the §11.6 tile header.

6.3 References

6.3.1 Related Documents

- **cell PRD** (../../../PRD.md) — product definition, normative for the cell
- **TILE_SPEC §11** (../../../../tile/docs/TILE_SPEC.md) — packet header, transport, credits/grants, coordinates
- **alu-infer PRD** (../../../../alu-infer/PRD.md) — normative definition of the ALU block
- **alu-infer MAS** (../../../../alu-infer/docs/alu_infer_mas/alu_infer_mas_index.md) — implementation-intent book for the ALU block
- **delta_mesh_cell** (../../../../../fabric-gen-ip/delta/rtl/delta_mesh_cell.sv) — the node the cell instantiates

---

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 Design Philosophy and Scope

7.1 What This Block Is

The **cell** is the tile's compute unit: sixteen of them make a tile. Each cell contains exactly one `delta_mesh_cell` node, instantiated inside `cell_top`, with its local port (port index 8, `P_LOCAL`) facing the cell's internal datapath and its eight mesh-facing ports passing through to the cell boundary. That containment is the entire cell↔delta relationship: the node is governed by delta's own HAS/MAS, and everything else in the cell is the cell's own design, speaking the tile-level contracts of `TILE_SPEC` §11.

The cell's job is to turn delta-delivered operand lines into result lines that leave back through the delta mesh toward RAPIDS:

1. **Program.** `cfg` packets write the §7 registers — `FUNC_SEL`, `OP_SRC`, batching knobs, and the persistent operands `A`–`D`, `T`.
2. **Fill.** `mem` packets write operand lines into the input memory, each line tagged with the packet's source coordinate.
3. **Consume.** The execution FSM pops `OP_LINES` input lines per invocation, issues them to the ALU, and pushes result lines into the result memory.
4. **Credit return.** As each input line is consumed, the credit logic returns credits to the recorded source.
5. **Emit.** The packetizer batches result flits into packets and sends them toward the gatling-programmed drain destination, spending the cell's `TILE_LAND_FLITS` strap counter.

Everything — data, config, and programming — arrives in-band on delta (PRD §3; TILE_SPEC §11.13; alu-infer PRD §5).

7.2 Scope and Deferrals

In scope for this MAS:

- The slice-1 RTL as built: `cell_top`, `cell_rx`, `cell_cfg_regs`, `cell_mem_in`, `cell_mem_out`
- The slice-2/3 modules specified here: `cell_credit`, `cell_packetizer`, `cell_exec_fsm`
- The ALU↔FSM seam (the ALU itself is specified in the alu-infer MAS)
- The credit-loop and result-packetization behavior
- The verification plan for the credit-loop and ALU-run tests

Out of scope or deferred (PRD §10–11):

- Tetrad/root node internals and the tetrad control core
- RAPIDS internals and the CXS migration sequencing
- Fabric-level Z stacking
- Cell-destined result sending (`DEST` override) pending the tile-spec cell→cell grant extension
- Loading `A`–`D` from the input memory
- Wider-than-32-bit lines and element widths other than bf16

7.3 MAS Decisions at a Glance

This book closes the PRD §12 open questions with the following implementation decisions, each marked “MAS decision” with rationale in the chapter where it lives:

Table 1.1: Summary of MAS decisions

Decision	Value	Rationale
Input memory depth (`IN_DEPTH`)	32 lines	Matches result memory depth; gives a 32-flit advertisement today; simple power-of-two addressing
Result memory depth (`OUT_DEPTH`)	32 lines	Sized to absorb one full input run’s results at the default `OP_LINES`=1; deeper than the default `TILE_LAND_FLITS` strap so the memory is not the emit bottleneck
Line width (`LINE_W`)	32 bits (2×bf16)	bf16 element model per alu-infer PRD §2; slice-3 may pack two lines per 64-bit flit
Boot advertisement	32 flits	One 32-bit line per 64-bit payload flit today; the value is `IN_DEPTH` expressed in flits
Credit batch size (`CREDIT_BATCH_N`)	Default 1	Exact per-line behavior; configurable for lower

Decision	Value	Rationale
		fabric traffic
Partial-batch credit timeout	64 cycles	Bounded drain latency without adding credit traffic at N=1
Result batch size (`EMIT_BATCH_X`)	Configurable, default 4	Amortizes header overhead while respecting the 16-flit strap
Packetizer flush conditions	Batch-full OR `CTRL.flush-and-send`	Simple, deterministic; idle-timeout and end-of-job flush are deferred
Strap counter default	16 flits	Mirrors the Hive default `TILE_LAND_FLITS`; restored by RAPIDS `credit/add` returns
Local-port data width (slice 2/3)	Delta `DATA_WIDTH` (64b today)	Beat-chained header rule (owner ruling 2026-10-18): the 128-bit tile header spans the first `ceil(128/DW)` payload beats, LSB-first; no widening, no width conversion. Supersedes the 0.10 draft's 128-bit local port

7.4 Relationship to the ALU

The cell owns the memories, the FSM, the packetizer, and the credit logic. The ALU block — `alu-infer` — is the cell's compute engine and is governed by its own PRD and MAS. The cell MAS cites the alu-infer contracts at the FSM seam and does not restate the ALU internals.

Last Updated: 2026-10-18

8 Module Tree

8.1 Purpose

`cell_top` is the compute cell top-level. It instantiates one `delta_mesh_cell` node and the cell's own datapath blocks. The module tree below shows the slice-1 blocks that are built and the slice-2/3 blocks that this MAS specifies.

8.2 Module Tree

```
cell_top (built, slice 1)
├── delta_mesh_cell (black box — fabric-gen-ip/delta)
├── cell_rx (built, slice 1) — header parse / demux from the node's
local port
├── cell_cfg_regs (built, slice 1) — AXIL slave + in-band cfg-stream
register file
├── cell_mem_in (built, slice 1) — input operand memory with per-line
source tag
├── cell_mem_out (built, slice 1) — result memory
├── cell_credit (specified, slice 2) — per-source owed counters +
credit packet builder
├── cell_packetizer (specified, slice 2) — result-flit batching +
§11.5 header builder
└── cell_exec_fsm (specified, slice 3) — pop → issue → push loop + ALU
hookup
```

The ALU block itself (`alu-infer`) is instantiated inside `cell_exec_fsm` (or as a sibling driven by it); its specification is owned by the alu-infer PRD and MAS.

8.3 Block Intent (one paragraph each)

8.3.1 `delta_mesh_cell`

The network attachment. The cell instantiates one node; its local port (`P_LOCAL = 8`) delivers traffic to `cell_rx`, and its eight mesh ports pass through to the cell boundary. The node is a black box owned by `fabric-gen-ip/delta`; its HAS/MAS govern it. The cell depends only on the node's local-port AXI4-Stream contract and on the node's ability to route packets by destination coordinate.

8.3.2 `cell_rx`

Header parse and demux for the node's local port. Slice 1 already parses a provisional 64-bit TUSER packing (because the instantiated delta node exposes a 64-bit TUSER) and routes `mem` payloads to the input memory, `cfg` writes to the register file, and `credit` packets to a counted TODO drop. Slice 2/3 reworks `cell_rx` to the §11.6 transport: delta's route header rides TUSER,

and the 128-bit tile header is the first data beat. Every input-memory line is tagged with the packet's source coordinate so credit returns can be addressed correctly.

8.3.3 cell_cfg_regs

AXI4-Lite slave plus in-band `cfg`-stream register file per PRD §7. The register map is defined in Chapter 6. The block exposes register values to the execution FSM and packetizer, generates the one-cycle `soft_rst` pulse, and assembles `STATUS` and `IRQ_STATUS`.

8.3.4 cell_mem_in

Input operand memory. Sized to `IN_DEPTH` lines; each line carries a source-coordinate tag. It has one write port (from `cell_rx`) and one read port (from `cell_exec_fsm`). The read pointer `IP` is owned by the FSM; occupancy is exposed for `STATUS`.

8.3.5 cell_mem_out

Result memory. Sized to `OUT_DEPTH` lines; one write port (from `cell_exec_fsm`) and one read port (from `cell_packetizer`). The write pointer `OP` is owned by the FSM; occupancy is exposed for `STATUS`.

8.3.6 cell_credit

Credit-return logic. Maintains per-source owed counters keyed by the source coordinate recorded on each consumed input line. Batches returns to `N = CREDIT_BATCH_N` consumed lines per source; drains a partial batch after a 64-cycle timeout. Builds header-only `credit`/`add` packets addressed to the recorded source and sends them back through the node on the credit VC. Sends the boot `credit`/`set` advertisement to `level` 3 carrying the input memory's size in flits.

8.3.7 cell_packetizer

Result drain and packetizer. Batches result flits up to `EMIT_BATCH_X`; flushes on batch-full or on `CTRL.flush-and-send`. Builds the §11.5 tile header with destination and tag supplied by the gatling-programmed drain command, and sends the packet through the node only while the `TILE_LAND_FLITS` strap counter covers the whole packet. Spends the strap counter per flit sent (header included) and restores it as RAPIDS returns `credit`/`add` packets.

8.3.8 cell_exec_fsm

Execution FSM and ALU hookup. Runs the pop→issue→push loop: pops `OP_LINES` input lines per invocation (advancing `IP`), presents them to the ALU's valid/ready elastic pipeline, and pushes each retired result line into the result memory (advancing `OP`). Selects operand sources per `OP_SRC` (input-memory pop or one of `A`/`B`/`C`/`D`; `T` backs the threshold immediate). Clears `IP`/`OP` on `CTRL.soft_reset`.

8.4 Parameters

Table 2.1: `cell_top` parameters

Parameter	Default	Meaning
`LINE_W`	32	Line width in bits; one line is two bf16 lanes
`COORD_W`	4	Coordinate sub-field width; coordinate is `{ext, level, X, Y, Z}`
`POS_X/Y/Z`	0	This cell's position in the tile grid
`IN_DEPTH`	32	Input memory depth in lines
`OUT_DEPTH`	32	Result memory depth in lines
`AXIL_ADDR_WIDTH`	32	AXIL address width
`AXIL_DATA_WIDTH`	32	AXIL data width
`DATA_WIDTH`	64	Cell↔delta local-port data width. Stays at the delta node's `DATA_WIDTH` (64 today); the 128-bit §11.6 tile header is beat-chained LSB-first across `ceil(128/DATA_WIDTH)` beats. No local-port widening.
`USER_WIDTH`	64	Delta route header width; unchanged until delta widens it
`NUM_VC`	2	Number of delta virtual channels

8.5 Clock and Reset

Single clock domain, active-low async `aresetn` (house `ALWAYS_FF_RST`), matching the tile's single-clock-domain decision (TILE_SPEC §9). Reset clears the memories, registers, pointers, and credit/strap state; `CTRL.soft_reset` clears only `IP`, `OP`, `IRQ_STATUS`, and selected datapath state.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Datapath Figures

9.1 Whole-Cell Datapath

9.1.1 Figure 2.1: Whole-cell datapath

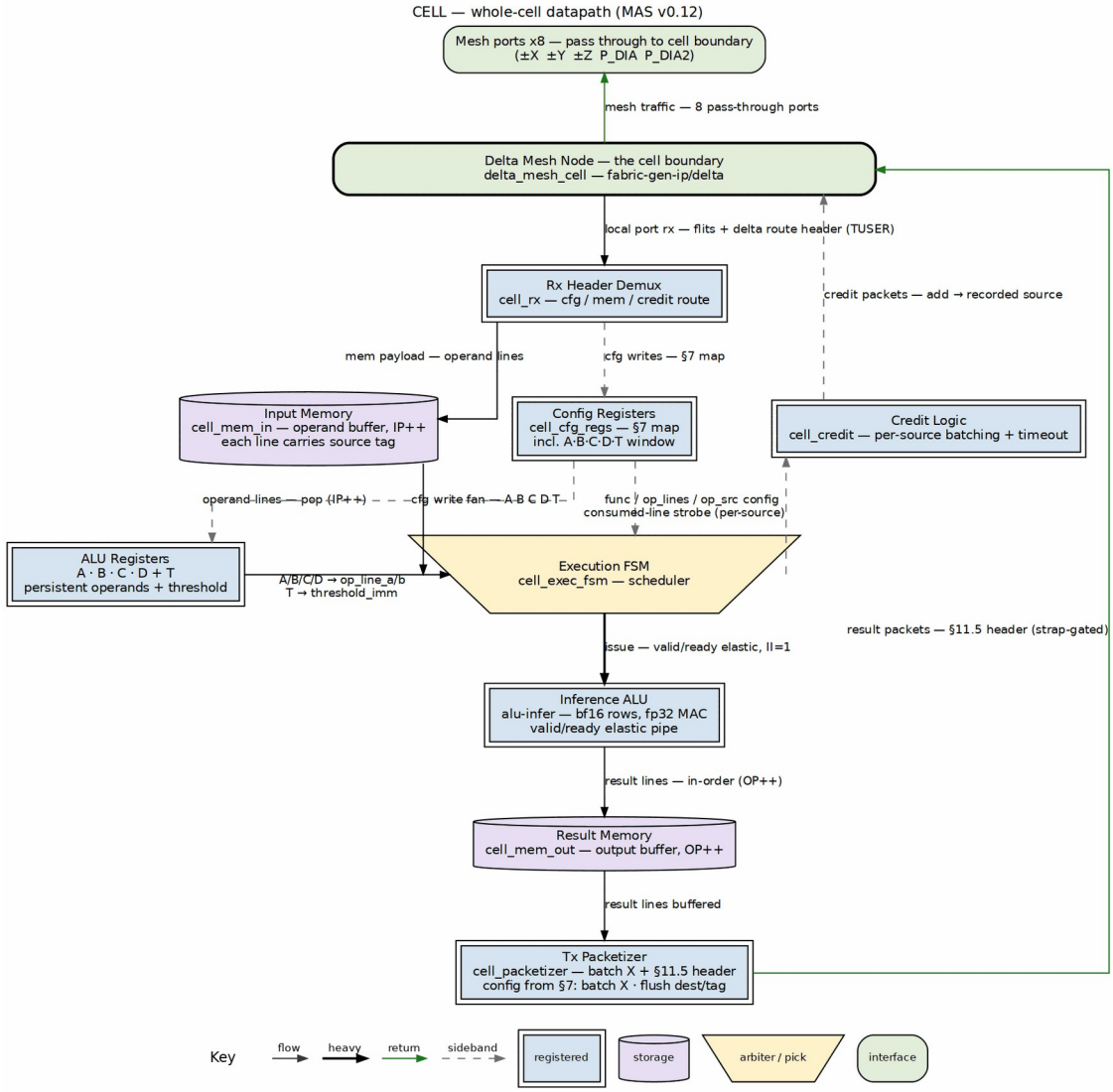


Figure 2.1: Whole-cell datapath

The figure follows the house-style rule that three flows must be traceable at a glance:

- **Programming/config flows down the center:** `cfg` packets enter through the delta node, are demuxed by `cell_rx`, and write the config registers; the registers in turn configure the FSM, the ALU, and the packetizer.

- **Operand fill flows down the left:** `mem` payloads are written into the input memory and are later popped by the execution FSM.
- **Results and credit returns flow back up the right:** result lines move from the ALU into the result memory, through the packetizer, and back out through the delta node; consumed-line credits return on their own dashed path up to the node.

The boundary bar is the delta node edge. Every arrow is labeled with the transaction class it carries; sideband/config arrows are dashed.

9.2 Credit Loop

9.2.1 Figure 2.2: Credit loop

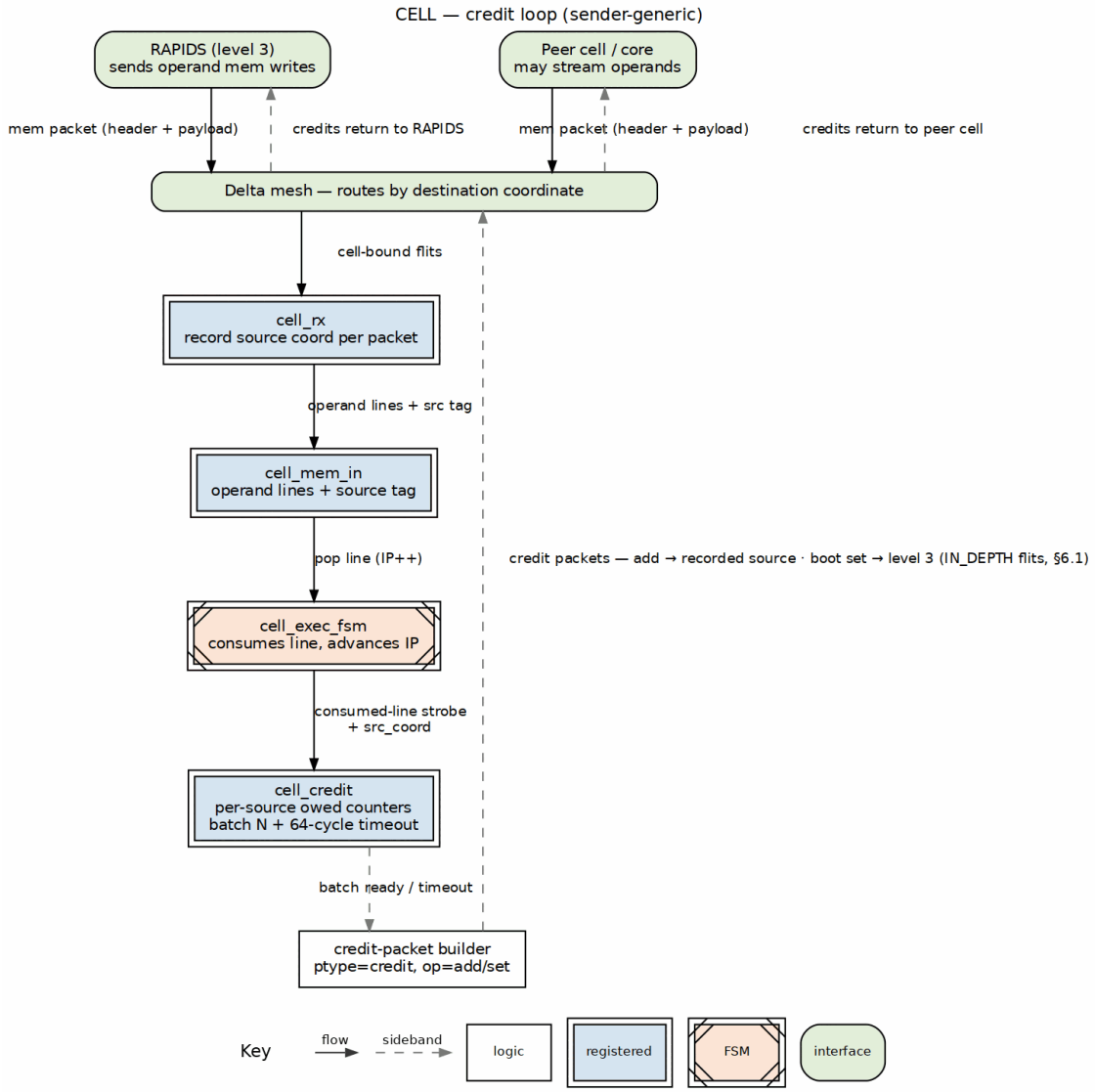


Figure 2.2: Credit loop

The credit-loop figure shows the sender-generic return path:

1. An input line arrives from a source and is tagged with that source's coordinate.
2. The execution FSM consumes the line and strobes `cell_credit`.

3. `cell_credit` increments a per-source owed counter and, when the batch threshold `CREDIT_BATCH_N` is reached (or the 64-cycle timeout expires), sends a header-only `credit`/`add` packet addressed to the recorded source.
4. The packet travels back through the delta node to the source, which restores its per-destination credit counter.

The path never shares an arrow with result data; credits are a separate sideband VC.

9.3 ALU Datapath Focus

9.3.1 Figure 2.3: ALU datapath focus

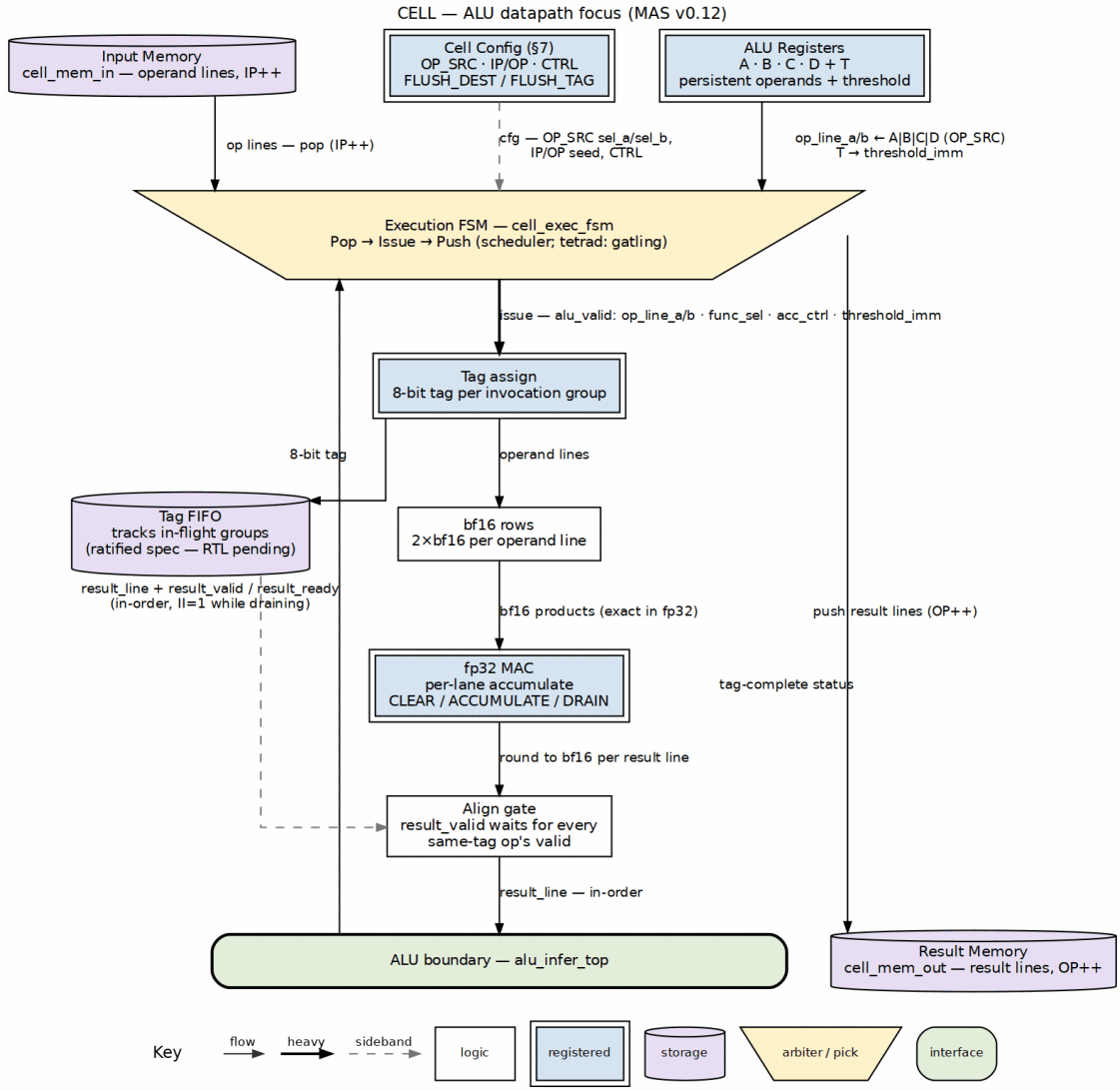


Figure 2.3: ALU datapath focus

The ALU-focus figure drops the delta/network side and traces the three cell-internal flows around `alu_infer_top`:

1. **Operand fill (left):** `cell_mem_in` pops (`IP++`) and the persistent `A·B·C·D/T` registers feed the execution FSM, which owns the `OP_SRC` operand mux. Config (`OP_SRC`, `IP/OP`, `CTRL`) arrives dashed from §7.
2. **Issue (center, heavy pen):** the FSM issues the bundle (`op_line_a/b`, `func_sel`, `acc_ctrl`, `threshold_imm`) with `alu_valid`; the tag assign gives the invocation group its 8-bit tag and the tag FIFO tracks in-flight groups (ratified spec — RTL pending). Rows multiply in bf16 exactly, the fp32 MAC accumulates per-lane under `acc_ctrl`, and the align gate holds `result_valid` until every same-tag op's valid has asserted.
3. **Result flow (right):** `result_line` returns **up** its own route through the ALU boundary bar (`result_valid`/`result_ready`, in-order, II=1 while draining), and the FSM pushes the accepted lines down to `cell_mem_out` (`OP++`).

The single boundary bar is the `alu_infer_top` edge; no flow shares an arrow with another.

Last Updated: 2026-10-18

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 External and Internal Interfaces

10.1 Cell Boundary

10.1.1 Mesh-Facing Ports

`cell_top` exposes the eight mesh-facing ports of the instantiated `delta_mesh_cell` node:

Table 3.1: Mesh-facing AXI4-Stream ports

Port	Direction	Width	Purpose
`s_axis_tdata[8]`	in	`DATA_WIDTH`	Data from mesh neighbors
`s_axis_tvalid[8]`	in	1	Valid from mesh neighbors
`s_axis_tready[8]`	out	1	Ready to mesh neighbors
`s_axis_tlast[8]`	in	1	Packet last from mesh neighbors
`s_axis_tuser[8]`	in	`USER_WIDTH`	Delta route header from mesh neighbors
`m_axis_tdata[8]`	out	`DATA_WIDTH`	Data to mesh neighbors
`m_axis_tvalid[8]`	out	1	Valid to mesh neighbors
`m_axis_tready[8]`	in	1	Ready from mesh neighbors
`m_axis_tlast[8]`	out	1	Packet last to mesh neighbors
`m_axis_tuser[8]`	out	`USER_WIDTH`	Delta route header to mesh neighbors
`s_credit[8]`	in	`NUM_VC`	Per-VC credit levels from neighbors
`m_credit[8]`	out	`NUM_VC`	Per-VC credit

Port	Direction	Width	Purpose
			levels to neighbors

These ports are pass-through wiring in `cell_top`; the node performs all routing. Cell-bound traffic must be addressed to the cell's external ID `{ext=1, level=0, X, Y, Z}` and ride delta QoS ≠0 so the node does not consume it as its own config traffic (TILE_SPEC §11.2).

10.1.2 Local Port Contract

The cell's internal datapath sees only the node's local port (`P_LOCAL = 8`). Slice 1 used a provisional 64-bit TUSER packing for the tile header because the instantiated delta node exposes 64-bit `TUSER`.

MAS decision: Slice 2/3 moves to the `TILE_SPEC` §11.6 transport binding, with the beat-chained header rule (owner ruling 2026-10-18): every link keeps its own data width, and the 128-bit tile header occupies the first `ceil(128/DW)` payload beats, LSB-first.

- The **delta route header** rides `TUSER` of the head beat, as delta owns it.
- The **128-bit tile header** is payload: on the 64-bit local port it spans beats 0–1 (`tdata[63:0]` = header `[63:0]`, next beat's `tdata[63:0]` = header `[127:64]`), then payload beats follow. On a 128-bit-or-wider link it is beat 0 alone.
- The cell's local-port data width **stays at the delta node's `DATA_WIDTH` (64 bits today)** — no widening, no width conversion anywhere. The demux reassembles the two header beats; the packetizer emits the header across two beats.

This supersedes slice-1's provisional 64-bit TUSER packing and the earlier MAS 0.10 draft's 128-bit local-port widening.

10.1.3 AXI4-Lite Register Interface

The cell exposes a house-standard AXI4-Lite slave for debug and local control. All registers are also writable from the in-band `cfg` stream. The register map is in Chapter 6.

10.1.4 Observability

Table 3.2: Observability outputs

Signal	Width	Purpose
`o_err_drop_cnt`	32	Dropped-head count from the delta node
`o_todo_credit_drop_cnt`	32	Slice-1 TODO counter for

Signal	Width	Purpose
		unhandled credit packets
`o_todo_cfg_read_drop_cnt`	32	Slice-1 TODO counter for unhandled cfg read replies
`o_irq`	1	Interrupt output (level)
MonBus (AXI4-Lite)	-	The cell's one MonBus stream, merged at the tetrad with the other three cells and the tetrad node (tile spec §5). Not yet in the RTL

10.2 Internal Interfaces

10.2.1 `cell_rx` → `cell_mem_in`

Table 3.3: Input-memory write port

Signal	Width	Direction	Purpose
`mem_wr_valid`	1	rx → mem	Write valid
`mem_wr_ready`	1	mem → rx	Write ready
`mem_wr_addr`	`log2(IN_DEPTH)`	rx → mem	Line index
`mem_wr_data`	`LINE_W`	rx → mem	Line data
`mem_wr_src_coord`	`COORD_BITS`	rx → mem	Source coordinate tag

10.2.2 `cell_rx` → `cell_cfg_regs`

Table 3.4: Config-stream register interface

Signal	Width	Direction	Purpose
`cfg_wr_valid`	1	rx → regs	In-band cfg write valid
`cfg_wr_ready`	1	regs → rx	In-band cfg write ready
`cfg_wr_addr`	8	rx → regs	Register offset
`cfg_wr_data`	32	rx → regs	Register value

Signal	Width	Direction	Purpose
cfg_rd_valid	1	rx → regs	In-band cfg read request valid (slice-2 completion)
cfg_rd_addr	8	rx → regs	Register offset
cfg_rd_data	32	regs → rx	Read data (slice-2 completion)
cfg_rd_ready	1	rx → regs	Read request accepted (slice-2 completion)

10.2.3 cell_cfg_regs → datapath

The register block drives the following register values:

Table 3.5: Register outputs to datapath

Signal	Width	Purpose
r_ctrl	32	CTRL register
r_func_sel	32	FUNC_SEL register
r_op_lines	32	OP_LINES register
r_emit_batch_x	32	EMIT_BATCH_X register
r_credit_batch_n	32	CREDIT_BATCH_N register
r_irq_en / r_irq_status	32	IRQ enables and status
r_a / r_b / r_c / r_d / r_t	32	Persistent ALU operands and threshold
r_op_src	32	Operand-source mux selection
r_dest	32	Deferred cell-destined destination
r_ip / r_op	32	Input/result memory pointers
o_soft_rst	1	One-cycle soft-reset pulse

10.2.4 `cell_mem_in` → `cell_exec_fsm`

Table 3.6: Input-memory read port

Signal	Width	Direction	Purpose
`mem_in_rd_valid`	1	fsm → mem	Read request valid
`mem_in_rd_ready`	1	mem → fsm	Read request ready
`mem_in_rd_addr`	`log2(IN_DEPTH)`	fsm → mem	Read address
`mem_in_rd_data`	`LINE_W`	mem → fsm	Read data (registered)
`mem_in_rd_src_coord`	`COORD_BITS`	mem → fsm	Source tag of read line
`mem_in_empty`	1	mem → fsm/reg	Memory empty
`mem_in_full`	1	mem → fsm/reg	Memory full
`mem_in_occupancy`	`log2(IN_DEPTH+1)`	mem → fsm/reg	Line occupancy

10.2.5 `cell_exec_fsm` → `cell_mem_out`

Table 3.7: Result-memory write port

Signal	Width	Direction	Purpose
`mem_out_wr_valid`	1	fsm → mem	Write valid
`mem_out_wr_ready`	1	mem → fsm	Write ready
`mem_out_wr_addr`	`log2(OUT_DEPTH)`	fsm → mem	Write address
`mem_out_wr_data`	`LINE_W`	fsm → mem	Result line

10.2.6 `cell_exec_fsm` → `cell_credit`

Table 3.8: Consumed-line credit strobe

Signal	Width	Direction	Purpose
`credit_strobe`	1	fsm → credit	One input line consumed
`credit_src_coord`	`COORD_BITS`	fsm → credit	Source coordinate of consumed line

10.2.7 `cell_credit` → node (credit VC)

`cell_credit` builds header-only `credit/add` and `credit/set` packets and injects them into the node's local-port credit sideband. The packet format is defined in TILE_SPEC §11.5 and §11.9.

10.2.8 `cell_mem_out` → `cell_packetizer`

Table 3.9: Result-memory read port

Signal	Width	Direction	Purpose
`pkt_rd_valid`	1	pkt → mem	Read request valid
`pkt_rd_ready`	1	mem → pkt	Read request ready
`pkt_rd_addr`	`log2(OUT_DEPTH)`	pkt → mem	Read address
`pkt_rd_data`	`LINE_W`	mem → pkt	Result line
`mem_out_empty`	1	mem → pkt	Memory empty
`mem_out_occupancy`	`log2(OUT_DEPTH+1)`	mem → pkt	Line occupancy

10.2.9 `cell_packetizer` → node (data network)

`cell_packetizer` builds write-format packets (`ptype=mem`, `op=0`) with the §11.5 tile header and sends them through the node's local port. The destination and tag are supplied by the gatling-programmed drain command; the source coordinate is the cell's external ID.

10.2.10 `cell_exec_fsm` ↔️ ALU

The ALU interface is defined in the alu-infer MAS; the cell MAS cites it in Chapter 4.3.

Last Updated: 2026-10-18

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Cycle Behavior: Fill Path as Built

11.1 Slice-1 Fill Path

The slice-1 RTL already implements the receive and store side of the cell. This section documents its cycle behavior so that the slice-2/3 work can be checked against it.

11.1.1 Head-Beat Reception

A packet arrives on the node's local port as a sequence of AXI4-Stream beats. On the head beat:

- `s_user[11:0]` carries the destination `{X, Y, Z}`.
- `s_user[13:12]` carries the QoS class; it must be `2'b01` (non-zero) so the node does not consume the packet as its own config traffic.
- `s_user[29:15]` carries the source coordinate `{ext, level, X, Y, Z}`.
- `s_user[32:30]` carries `ptype`: `cfg=0`, `mem=2`, `credit=3`, ...
- `s_user[33]` carries `op` (write/read or credit add/set).
- `s_user[49:34]` carries a compacted 16-bit length.
- `s_user[59:50]` carries a compacted 10-bit target address.
- `s_user[63:60]` carries the 4-bit tag.

This 64-bit TUSER packing is provisional and exists only because the instantiated delta node exposes a 64-bit `TUSER`. It is superseded in slice 2/3 by the `TILE_SPEC` §11.6 two-header model.

11.1.2 Coordinate Match

`cell_rx` compares the destination `{X, Y, Z}` against its own `POS_X/Y/Z`. The `ext` bit and QoS are not part of the match; QoS is already required to be non-zero so the node did not consume the packet.

11.1.3 Demux Decision

Table 4.1: Slice-1 head-beat dispatch

ptype	op	Action
`mem`	0 (write)	Accept payload beats into `cell_mem_in` starting at `target_addr`; increment `mem_wr_ptr` per beat
`cfg`	0 (write)	Drive `cfg_wr_valid` to `cell_cfg_regs` with

ptype	op	Action
		target_addr[7:0] and s_data[31:0]
cfg	1 (read)	Counted TODO drop in slice 1; slice 2 builds a reply packet
credit	any	Counted TODO drop in slice 1; slice 2 routes to cell_credit
other	any	Drop head; increment o_err_drop_cnt via inc_err_drop

11.1.4 Payload Write Timing

For a mem write packet:

- Cycle 0: head beat accepted; in_pkt latches pkt_src_coord, pkt_ptype, pkt_target; mem_wr_ptr clears.
- Cycle N: each payload beat with s_valid && s_ready && mem_wr_ready writes mem_data[addr] and mem_tag[addr] and increments mem_wr_ptr, where addr = pkt_target + mem_wr_ptr.
- The last payload beat (s_last) clears in_pkt.

cell_mem_in exposes wr_ready = !o_full; cell_rx backpressures the local port while a mem payload is in flight and the memory is full.

11.1.5 Occupancy Accounting

cell_mem_in increments its occupancy counter on every accepted write and decrements it on every accepted read. The read side is held idle in slice 1 (mem_in_rd_valid = 1'b0 in cell_top), so occupancy only rises during fill tests.

11.2 Expected Slice-2/3 Cycle Behavior

Slice 2/3 keeps the same fill timing but changes the header transport:

- The head beat's TUSER carries only the delta route header.
- The 128-bit tile header is **payload**, beat-chained per the owner ruling 2026-10-18: on the 64-bit local port, beat 0 carries tdata = header[63:0] and beat 1 carries tdata = header[127:64]; payload beats follow. The demux reassembles the header across the two beats before dispatching on ptype.

- Payload beats carry one 32-bit line per 64-bit flit today; slice 3 packs two lines per 64-bit flit.

The demux still records the source coordinate from the tile header's source field (not delta's hop-source route header) and tags each input-memory line with it.

Last Updated: 2026-10-18

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 Credit Logic

12.1 MAS Decision: Memory Sizing and Advertisement

MAS decision: `IN_DEPTH = 32` lines and `OUT_DEPTH = 32` lines.

Rationale: A 32-line depth is a power of two, matches the default credit-strap scale, and gives simple 5-bit pointers. The result memory is intentionally deeper than the default `TILE_LAND_FLITS` strap (16 flits) so that the landing-space strap, not the result memory, is the emit bottleneck. The input memory's flit capacity is the cell's boot advertisement value.

With `LINE_W = 32` and a 64-bit payload flit today, one input line occupies one payload flit (low 32 bits). Therefore the boot advertisement is:

```
advertisement flits = IN_DEPTH = 32 flits
```

Note: Slice 3 may pack two 32-bit lines per 64-bit payload flit; if that packing is adopted, the advertisement value must be re-derived from the new flit count (`IN_DEPTH / 2` flits) while the line count stays 32. Until then, the advertisement is 32 flits.

12.2 MAS Decision: Credit Batch Size and Timeout

MAS decision: Per-source credit batching uses `N = CREDIT_BATCH_N` consumed lines per source, with a default of `N = 1`. A partial batch that has not reached `N` drains after a 64-cycle timeout.

Rationale: `N = 1` gives the simplest exact-per-line behavior, which is what the slice-2 credit-loop test will verify. Making `N` configurable lets later builds trade return latency for lower credit-packet traffic. The 64-cycle timeout bounds the latency of a trailing partial batch without adding any traffic at `N = 1`.

12.3 Per-Source Owed Counters

`cell_credit` maintains a bank of owed counters indexed by the recorded source coordinate. Each counter has:

- `owed[src]`: number of consumed lines not yet returned to this source
- `timer[src]`: cycles since the last return to this source (valid when `owed` is non-zero and below the batch threshold)

On every consumed-line strobe from `cell_exec_fsm`:

1. `owed[src] += 1`
2. If `owed[src] >= CREDIT_BATCH_N`, send one `credit/add` packet for `CREDIT_BATCH_N` flits, decrement `owed[src]` by `CREDIT_BATCH_N`, and clear the timer.

3. Otherwise, load the timer with the timeout value (or reset it if it was already running).

Every cycle, for each source with `0 < owed[src] < CREDIT_BATCH_N`:

- If `timer[src] == 0`, send one `credit`/`add` packet for `owed[src]` flits and clear `owed[src]`.
- Else, `timer[src] -= 1`.

The timeout value is 64 cycles. The timer is loaded only when a new partial batch begins; additional consumed lines in the same partial batch do not reset the timer (they will be included in the same return when it fires).

12.4 Credit Packet Format

A consumed-line credit return is a header-only `credit` packet with:

- `ptype = credit` (3)
- `op = 0` (add)
- `length` = number of flits returned (`CREDIT_BATCH_N` or the residual `owed` count)
- `destination` = the recorded source coordinate of the consumed line
- `source` = this cell's external ID
- `tag` = echoed or zero (the sink does not store credit packets)
- `target_address[1:0] = credit_class = 0` (sink space)

Credit packets ride the credit VC and need no grant (TILE_SPEC §11.9). TILE_SPEC §11.5 encodes the credit class in the low two bits of the target address: `0` = sink-space credit, `1` = descriptor-slot credit. Consumed-line returns and the boot advertisement below are sink-space credits (class 0).

12.5 Boot Advertisement Packet

After reset, `cell_credit` sends one `credit`/`set` packet to `level = 3` (the tile I/O / RAPIDS) carrying the input memory's full size in flits:

- `op = 1` (set)
- `length = IN_DEPTH` flits (32 today)
- `destination = {ext=0, level=3, X/Y/Z = tile I/O}`
- `source` = this cell's external ID
- `target_address[1:0] = credit_class = 0` (sink space)

This advertisement is the cell's "alive" signal; RAPIDS's external sink-space counter for this cell starts at 0 and is filled by this packet (TILE_SPEC §11.10).

12.6 Sender-Generic Returns

The return is sender-generic: a credit goes to the source coordinate recorded on the input-memory line, not to a hard-coded RAPIDS ID. A cell that streamed into this cell's input memory therefore receives the credits, not RAPIDS, exactly as TILE_SPEC §11.9 requires.

12.7 Implementation Note: Counter Bank Sizing

The number of legal source coordinates in a tile is bounded by the credit-counter index table in TILE_SPEC §11.9. A cell therefore needs at most enough per-source owed counters to cover every possible source that may send to it. A practical implementation allocates one counter per external ID (RAPIDS + peer cells + tetrad cores) that the tile can address, lazily allocating on first strobe if area must be minimized.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 Packetizer and Execution FSM

13.1 Packetizer

13.1.1 MAS Decision: Batching and Flush Conditions

MAS decision: The packetizer batches result flits up to `EMIT_BATCH_X` and flushes on either of two conditions:

1. The batch accumulator holds `EMIT_BATCH_X` flits.
2. `CTRL.flush-and-send` is written (bit 2 of `CTRL`).

An idle-timeout flush and an end-of-job marker flush are documented as deferred (PRD §12; PRD §11). This keeps slice-2 behavior deterministic and testable: a testbench can either fill a batch or force a flush.

13.1.2 MAS Decision: Strap Counter Initialization

MAS decision: The packetizer maintains a strap counter initialized from a cell parameter `TILE_LAND_FLITS` with a default of 16 flits. The counter is spent per flit sent (header included) and is restored by RAPIDS `credit`/`add` returns.

Rationale: The default 16 matches the Hive-set `TILE_LAND_FLITS` default in TILE_SPEC §11.9. The counter is private to this cell and never goes negative; a packet is presented only while the counter covers the entire packet, header included.

13.1.3 Header Build

The packetizer builds write-format packets (`ptype=mem`, `op=0`) with the `TILE_SPEC` §11.5 tile header as the first payload beat(s). The header fields are:

Table 4.2: Result-packet header fields

Field	Value
Destination coordinate	Gatling-programmed drain destination (see below)
Source coordinate	This cell's external ID `{ext=1, level=0, X, Y, Z}`
`ptype`	`mem` (2)
`op`	0 (write)
Length	Payload bytes that follow
Target address	Programmed by the drain command (memory address in the sink channel)

Field	Value
Tag	Programmed sink channel

MAS decision on transport: The packetizer uses the TILE_SPEC §11.6 two-header model with the beat-chained header rule (owner ruling 2026-10-18 — every link keeps its own width):

- The delta route header is placed in TUSER of the head beat.
- The 128-bit tile header is placed in the first `ceil(128/DW)` payload beats, LSB-first — on the 64-bit local port: beat 0 = `header[63:0]`, beat 1 = `header[127:64]`.
- Payload beats follow.

The local port does not widen. This supersedes slice-1's provisional 64-bit TUSER packing and the 0.10 draft's 128-bit local-port widening.

13.1.4 Destination and Tag Programming

The cell does not choose the result destination. Gatling (the tetrad engine) commands the drain and tells the cell where to send it (TILE_SPEC §11.14; PRD §6.3).

Precedence: The operational drain destination is held in FLUSH_DEST (offset 0x54) and the sink-channel tag in FLUSH_TAG (offset 0x58). These are programmed by gatling (via `cfg` write or an extended CTRL flush command) and are what the packetizer uses when `CTRL.flush-and-send` is written. The legacy DEST register (offset 0x38) is **reserved** for the deferred cell→cell result path; it is not used for rapids-bound drains. DEST becomes operational only after the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10); until then, result traffic is RAPIDS-bound and FLUSH_DEST/FLUSH_TAG take precedence.

Rationale: This gives gatling a concrete mechanism to command a flush without requiring a new packet type or a sideband interface, while keeping the DEST register available for the future peer-cell path.

13.1.5 Strap Accounting

Each packet costs one header flit plus one flit per payload line today. The packetizer checks:

```
strap_counter >= 1 + payload_flits
```

before presenting `tvalid`. It decrements the counter by one per flit sent. RAPIDS returns credits as it drains the cell's landing space; the returns are `credit`/`add` packets that increment the strap counter.

13.2 Execution FSM

13.2.1 Pop → Issue → Push Loop

`cell_exec_fsm` runs the compute loop:

1. **Pop.** When enabled and not busy, pop `OP_LINES` input lines from `cell_mem_in`, advancing `IP` by one per line. Capture the source coordinate of the first line (all lines of one invocation are expected to come from the same source; if they differ, the first line's source is used for credit return).
2. **Issue.** Present the operand lines to the ALU and assert the issue `valid`. Wait for `ready`; hold operands and `func_sel` stable while `ready` is low.
3. **Push.** When the ALU asserts `result_valid` and the FSM asserts `result_ready`, push the result line into `cell_mem_out`, advancing `OP` by one.

The loop is pipelined: while one invocation computes, the FSM can pop and issue the next, as long as the ALU's elastic pipeline accepts it (II=1 while draining).

13.2.2 MAS Decision: Operand-Source Mux

MAS decision: The operand-source mux is configured by `OP_SRC`:

Table 4.3: `OP_SRC` encoding

`OP_SRC` value	`op_line_a` source	`op_line_b` source
0	input-memory pop	input-memory pop
1	`A`	`A`
2	`B`	`B`
3	`C`	`C`
4	`D`	`D`

The two sides are controlled by independent 3-bit fields in `OP_SRC`: `sel_a[2:0]` and `sel_b[2:0]`. Reset value is 0 (both operands from memory). The `T` register supplies `threshold_imm` for opcode 6 (compare/threshold).

13.2.3 IP/OP Pointer Behavior

`IP` and `OP` are 32-bit registers that auto-increment on pop and push respectively. They wrap at `IN_DEPTH` and `OUT_DEPTH`:

```
IP_next = (IP + 1) mod IN_DEPTH
OP_next = (OP + 1) mod OUT_DEPTH
```

CTRL.soft_reset clears both pointers to 0 and clears IRQ_STATUS. Software may also seed them through AXIL or cfg writes.

13.2.4 Gatling as the Cell ALU Sequencer (owner ruling 2026-10-19)

MAS decision: The slice-3 cell_exec_fsm remains the standalone bring-up driver for the cell, preserving the same POP → issue → PUSH client contract. In the tetrad-integrated tile, **gatling** (the tetrad barrel-core engine) sequences the cell's alu-infer ALU directly:

- POP reads operand line(s) from the cell's input memory and presents them to the ALU issue handshake.
- PUSH retires result line(s) from the ALU and writes them into the cell's result memory.
- FLUSH programs FLUSH_DEST/FLUSH_TAG and triggers the packetizer drain.
- CFG/ECTX/KICK/WAIT carry tetrad-level context and control.

Gatling never originates config traffic that belongs in the cell's cfg stream; cell programming remains in-band per the standing owner ruling 2026-10-17. The gatling PRD v0.2 owns the Xcell instruction encodings (CFG, POP, PUSH, ECTX, FLUSH, KICK, WAIT).

13.2.5 ALU Hookup

The FSM drives the ALU's elastic valid/ready interface per the alu-infer MAS:

Table 4.4: ALU🔄FSM seam signals

Signal	Direction	Purpose
alu_valid	FSM → ALU	Issue valid
alu_ready	ALU → FSM	Issue ready
op_line_a	FSM → ALU	Operand A line
op_line_b	FSM → ALU	Operand B line
threshold_imm	FSM → ALU	bf16 threshold immediate (from T)
func_sel	FSM → ALU	{opcode, sub-op, reserved}
acc_ctrl	FSM → ALU	CLEAR / ACCUMULATE / DRAIN for MAC
result_line	ALU → FSM	Result line
result_valid	ALU → FSM	Result valid

Signal	Direction	Purpose
result_ready	FSM → ALU	Result ready

The function/opcode table is defined in alu-infer PRD §3 and implemented per the alu-infer MAS. The cell MAS does not restate it.

13.2.6 Issue Tag and Same-Tag Alignment (ratified spec — RTL pending)

MAS decision: When the scheduler loads a new function into the ALU pipeline, an **8-bit tag** is assigned, common to every op of that invocation group that must complete together. A **tag FIFO** tracks the in-flight groups. Each op's result_valid asserts only when *every* result_valid of ops carrying the same tag has asserted — so ops that were scheduled together finish together, and the elastic pipeline's per-op readiness is gated to the group boundary.

Rationale (owner ruling): if two ops are scheduled in the same clock they should finish at the same time, and the scheduler programs the valid/ready behavior it wants (including feedback cases) through this tag grouping rather than through pipeline-depth matching. This takes care of the elastic alignment naturally: the group is atomic at the result boundary even though individual ops may have different internal latencies.

Status: ratified with the alu-infer work at rev 0.50; **not yet in the alu-infer RTL or the alu-infer MAS** — the tag assign/FIFO/align gate of Figure 2.3 is the specified target, not the current implementation.

13.2.7 MAC Accumulation

MAC behavior is inherited from the alu-infer contract: the FMA computes the bf16 product exactly in fp32, accumulates per-lane in fp32, and rounds to bf16 once per result-line presentation. The FSM sequences acc_ctrl (CLEAR at layer boundaries, ACCUMULATE for steps, DRAIN for the final values) without changing the ALU interface.

13.2.8 Soft Reset

CTRL.soft_reset (bit 1) pulses o_soft_rst for one cycle. The FSM responds by clearing IP and OP and dropping any in-flight issue request. The ALU's reset behavior is defined in the alu-infer MAS.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 Verification Plan

14.1 House Test Grid

The cell testbench follows the house gate / func / full grid:

Table 5.1: House test grid

Level	Scope	Typical seeds	Runtime target
gate	Smoke: elaboration, reset, register reset values, basic fill	1 seed	< 1 min
func	Directed feature tests across seeds	8 seeds	~5 min
full	Longer randomized runs, corner cases, coverage closure	32+ seeds	~30 min

14.2 Existing Tests

`cell/dv/tests/test_cell.py` already covers slice-1 behavior:

1. Elaboration + reset + register reset values over AXIL
2. AXIL and cfg-stream register access
3. Input-memory fill from a `ptype=mem` packet
4. Bad-packet drop behavior

The delta node inside `cell_top` is treated as a black box; slice-1 simulation drives the local rx port through the `test_mode` injection hook (`dut.test_rx_*`) so the testbench can supply cell traffic without fighting the node's continuous driver.

14.3 Planned Slice-2 Credit-Loop Test

The slice-2 test verifies the credit return path independently of the datapath:

1. Drive `mem` packets into the cell from multiple recorded sources (using the test-mode injection hook or a BFM on the local port).
2. Force the execution FSM to consume lines without running the ALU (or run a passthrough opcode).

3. Check that `cell_credit` emits `credit/add` packets whose destination equals the recorded source coordinate and whose length matches the configured batching (`CREDIT_BATCH_N` or residual).
4. Verify the 64-cycle partial-batch timeout by sending a partial batch and waiting for the automatic drain.
5. Verify the boot `credit/set` advertisement to `level=3` carrying `IN_DEPTH` flits.

The test forces straps so that result emission does not gate credit returns.

14.4 Planned Slice-3 ALU-Run Test

The slice-3 test verifies the pop→issue→push loop end to end:

1. Program `FUNC_SEL`, `OP_SRC`, `OP_LINES`, and persistent operands `A`–`D`, `T` via `cfg` or AXIL.
2. Fill the input memory with known operand lines.
3. Enable the FSM and run until `STATUS.busy` clears or an IRQ fires.
4. Read the result memory (via the packetizer drain or AXIL window).
5. Check results against the alu-infer golden model per opcode.

Coverage: opcode (8) × special-value classes × lane position × accumulator clear/hold/drain corners.

14.5 Verification Hooks

Table 5.2: Verification hooks

Hook	Use
`test_rx_*` / `test_mode`	Inject cell traffic without instantiating the delta mesh
`inc_lines_in/out`, `inc_credits`, `inc_pkts`	Count events independently of datapath
`o_todo_credit_drop_cnt` / `o_todo_cfg_read_drop_cnt`	Confirm slice-1 stubs are replaced in slice 2
`STATUS` / `IRQ_STATUS`	Observe empty/full, error, and strap-starved states

14.6 Assertions and Checks

- Credit return must match lines consumed, per source, within the batch timeout.
- Strap counter must never go negative; packetizer `tvalid` implies sufficient strap for the whole packet.
- ALU results retire in issue order.
- `IP` and `OP` wrap correctly at their respective depths.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 Register Map and MAS Decisions

15.1 Register Map

The v1 register map is identical to PRD §7, with two MAS-added operational registers for the gatling-programmed drain command.

Table 6.1: Cell register map

Offset	Register	Notes
0x00	CTRL	enable, soft reset, flush-and-send
0x04	STATUS	busy, input-empty, result-nonempty, strap-nonzero (RTL alias status_grant_nonzero)
0x08	FUNC_SEL	opcode (5b) + sub-op (2b) + reserved (2b)
0x0C	OP_LINES	input lines per function invocation
0x10	EMIT_BATCH_X	flits per result packet
0x14	CREDIT_BATCH_N	consumed lines per credit packet per source
0x18	IRQ_EN	done, error, strap-starved (RTL alias IRQ_GRANT_STARVED_BIT) , credit-timeout enables
0x1C	IRQ_STATUS	write-1-clear status
0x20	A	persistent operand A
0x24	B	persistent operand B
0x28	C	persistent operand C
0x2C	D	persistent operand D
0x30	T	bf16 threshold immediate
0x34	OP_SRC	operand-source mux: sel_a[2:0], sel_b[2:0]

Offset	Register	Notes
0x38	DEST	deferred cell-destined destination
0x3C	IP	input-memory read pointer
0x40	OP	result-memory write pointer
0x44	CNT_LINES_IN	lines received
0x48	CNT_LINES_OUT	results produced
0x4C	CNT_CREDITS	credit packets returned
0x50	CNT_PKTS	result packets sent
0x54	FLUSH_DEST	**MAS-added:** gatling-programmed drain destination coordinate
0x58	FLUSH_TAG	**MAS-added:** gatling-programmed sink-channel tag

15.2 Bit Fields

15.2.1 CTRL (0x00)

Table 6.2: CTRL bit fields

Bit	Name	Purpose
0	EN	Enable execution FSM
1	SOFT_RST	Soft reset: clears IP, OP, IRQ_STATUS; pulse lasts one cycle
2	FLUSH	Packetizer flush-and-send: emit whatever is buffered now

15.2.2 STATUS (0x04) — read-only

Table 6.3: STATUS bit fields

Bit	Name	Purpose
0	BUSY	FSM busy (slice 3)
1	INPUT_EMPTY	Input memory empty
2	RESULT_NONEMPTY	Result memory not empty
3	STRAP_NONZERO	Strap counter > 0. Slice-1 RTL still names this bit STATUS_GRANT_NONZERO_BIT / status_grant_nonzero; the doc term is canonical.

15.2.3 FUNC_SEL (0x08)

Table 6.4: FUNC_SEL fields

Bits	Field	Purpose
4:0	opcode	ALU opcode (alu-infer PRD §3)
6:5	sub_op	Sub-opcode, e.g. min vs max, compare vs threshold
8:7	reserved	Reserved under the bf16-only model

15.2.4 OP_SRC (0x34)

Table 6.5: OP_SRC fields

Bits	Field	Encoding
2:0	sel_a	0=memory pop, 1=A, 2=B, 3=C, 4=D
5:3	sel_b	0=memory pop, 1=A, 2=B, 3=C, 4=D
31:6	reserved	0

15.2.5 IRQ_STATUS / IRQ_EN

Table 6.6: IRQ fields

Bit	Name	Purpose
0	DONE	Computation/drain complete
1	ERROR	Dropped packet or fatal protocol error
2	STRAP_STARVED	Packetizer has data but no strap. Slice-1 RTL still names this bit IRQ_GRANT_STARVED_BIT; the doc term is canonical.
3	CREDIT_TIMEOUT	Partial-batch credit timer fired (informational)

15.3 MAS Decisions Revisited

Table 6.7: MAS decisions and their hardware homes

Decision	Where in this book	Register / parameter
Input/result memory depth = 32 lines	Chapter 4.2	IN_DEPTH, OUT_DEPTH parameters
Boot advertisement = 32 flits	Chapter 4.2	Derived from IN_DEPTH
Credit batch default = 1	Chapter 4.2	CREDIT_BATCH_N register
Partial-batch timeout = 64 cycles	Chapter 4.2	Hard-coded in cell_credit
Result batch flush conditions	Chapter 4.3	EMIT_BATCH_X, CTRL.FLUSH
Strap default = 16 flits	Chapter 4.3	TILE_LAND_FLITS parameter
Local-port width stays at delta DATA_WIDTH (64 bits today)	Chapter 3, Chapter 4.3	DATA_WIDTH parameter
Gatling-programmed	Chapter 4.3	FLUSH_DEST, FLUSH_TAG

Decision	Where in this book	Register / parameter
drain		registers

15.4 RTL Naming Divergence (slice 1)

Slice-1 RTL (`cell_pkg.sv`, `cell_cfg_regs.sv`) predates the 2026-10-18 ruling that renamed result-path grants to straps. The canonical MAS/PRD terms are **strap counter**, `STRAP_NONZERO`, and `STRAP_STARVED`; the slice-1 RTL identifiers remain:

- `STATUS_GRANT_NONZERO_BIT` / `status_grant_nonzero` for `STATUS` bit 3
- `IRQ_GRANT_STARVED_BIT` for `IRQ_STATUS`/`IRQ_EN` bit 2

A future RTL pass (slice 2/3) should rename these to the strap terms. Until then, the register map above states the canonical names and notes the RTL aliases.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 References

16.1 Normative Specifications

- **cell PRD** — `projects/components/compute-eng-ip/cell/docs/PRD.md`.
  The governing product definition. The MAS implements PRD §12 open questions and cites §§3, 6, 7 throughout.

- **TILE_SPEC §11** — `projects/components/compute-eng-ip/tile/docs/TILE_SPEC.md`.
  Defines the packet header (§11.5), transport binding (§11.6), payload types (§11.2), destination coordinate (§11.3), end-to-end credits and straps (§§11.9–11.10), and result flush (§11.14).

- **alu-infer PRD §3** — `projects/components/compute-eng-ip/alu-infer/PRD.md`.
  Normative ALU function set, element model (bf16), and MAC accumulation semantics.

- **alu-infer MAS** — `projects/components/compute-eng-ip/alu-infer/docs/alu_infer_mas/alu_infer_mas_index.md`.
  Implementation-intent book for the ALU block: module tree, function rows, accumulator, ALU↔️FSM interface, and cycle behavior.

16.2 RTL Sources

- **cell_top.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_top.sv`.
  Slice-1 top-level: delta node, rx demux, registers, memories.

- **cell_rx.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_rx.sv`.
  Slice-1 header parse/demux with provisional 64-bit TUSER packing.

- **cell_cfg_regs.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_cfg_regs.sv`.
  Slice-1 AXIL + cfg-stream register file.

- **cell_mem_in.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_mem_in.sv`.
  Slice-1 input memory with per-line source tag.

- **cell_mem_out.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_mem_out.sv`.
  Slice-1 result memory.

- **cell_pkg.sv** — `projects/components/compute-eng-ip/cell/rtl/cell_pkg.sv`.
  Shared types, coordinate layout, register offsets, and slice-1 compact header offsets.

- **delta_mesh_cell.sv** — `projects/components/fabric-gen-ip/delta/rtl/delta_mesh_cell.sv`.
  The node the cell instantiates; defines the 9-port unpacked-array interface and `P_LOCAL = 8`.

16.3 Verification

- **test_cell.py** — `projects/components/compute-eng-ip/cell/dv/tests/test_cell.py`.
  Existing slice-1 cocotb test suite; the base for the slice-2/3 credit-loop and ALU-run tests.

16.4 House Style and Build

- **Block diagrams handbook** — `vault/handbook/authoring/block-diagrams.md`.
  Binding house style for the whole-cell and credit-loop figures.

- **alu-infer MAS PDF generator** — `projects/components/compute-eng-ip/alu-infer/docs/generate_mas_pdf.sh`.
  The build script this MAS generator is modeled on.

Last Updated: 2026-10-18